

University of Tübingen
Faculty of Science
Department of Computer Science

Master's Thesis Machine Learning

Self-supervised learning for light-sheet microscopy image segmentation

Carla López Martínez

02.03.2026

Reviewers

First Reviewer

Prof. Sven Nahnsen
(Biomedical Data Science)
Institute for Bioinformatics and
Medical Informatics
University of Tübingen

Second Reviewer

Prof. Nico Pfeifer
(Methods in Medical Informatics)
Institute for Bioinformatics and
Medical Informatics
University of Tübingen

López Martínez, Carla:

Self-supervised learning for light-sheet microscopy image segmentation

Master's Thesis Machine Learning

University of Tübingen

Thesis period: 01.09.2025 - 02.03.2026

ABSTRACT

Light-sheet microscopy is an emerging imaging technique that grants the ability to obtain 3D images of large samples, such as whole brains, at cellular and subcellular resolution at high speeds. The ability to quickly obtain such data is an important step in advancing research related to the diagnosis of diseases and the assessment of biological processes.

Artificial intelligence and machine learning models have been extensively used to aid bioimage analysis, but they are usually limited by the amount of annotated data they need to properly learn and perform well in tasks. Self-supervised learning emerged as a way to overcome this reliance on labels, being capable of matching or surpassing the performance of fully supervised models by using a fraction of the annotated data.

The application of self-supervised techniques to bioimage analysis is a popular topic of research, however most studies focus on macroscopic modalities such as magnetic resonance imaging or computed tomography, and the ones focusing on microscopic modalities mostly use 2D whole slide images. This creates a research gap where some bioimage modalities are well-studied while others are rarely mentioned.

This thesis explores the impact of self-supervised learning in applications involving the under researched 3D bioimage modality of light-sheet microscopy, specifically in segmentation tasks. The aim of this project is to provide insights into how different self-supervised approaches impact the performance of a model in light-sheet microscopy image segmentation, to serve as a guide to what works well and what does not.

For this purpose, a self-supervised training pipeline is developed, where the BYOL and masked image modelling techniques, combined with a SwinUNETR architecture, are tested on the Selma3D challenge microscopy dataset, with the BraTS MRI dataset serving as a benchmark to check if the pipeline components work as expected.

The results of the experiments conducted show that the chosen self-supervised learning approaches do not consistently improve downstream task performance, highlighting the importance of large-scale pre-training data and computational resources for effective self-supervised learning in this domain.

Key words: self-supervised learning, machine learning, bioimage, light-sheet microscopy, MRI, BYOL, masked image modelling, BraTS, Selma, SwinUNETR.

ACKNOWLEDGMENTS

I am incredibly grateful to my supervisor and primary reviewer Prof. Sven Nahnsen, who has granted me the opportunity to work on this exciting project and has guided me through all the obstacles and doubts that have arisen. Always patient and willing to help, I could not have done this project without him. I am also grateful to Luis Kuhn Cuellar and Carolin Schwitalla for their helpful advice and regular overseeing. Their support and the knowledge they have shared with me have been key to the completion of this thesis. I would also like to thank Prof. Nico Pfeifer for accepting the role of second reviewer for this project.

Finally, I would also like to thank my family and friends for always supporting me and helping me pursue my dreams. My parents and siblings especially have helped me keep my motivation high during this project and have pushed me to be the best version of myself, all of which have allowed me to finish this project successfully.

TABLE OF CONTENTS

1	INTRODUCTION	1
2	BACKGROUND	3
2.1	What is self-supervised machine learning?	3
2.2	Types of self-supervised learning	5
2.2.1	Contrastive learning	5
2.2.2	Non-contrastive learning	6
2.2.3	Generative learning	7
2.3	Bioimage modalities across biological scale	8
2.3.1	Medical macroscopy modalities	9
2.3.2	Microscopy modalities in biology	10
2.4	Self-supervised learning in bioimage applications	12
2.5	Popular deep-learning architectures for bioimage analysis	14
2.5.1	Convolutional neural networks	14
2.5.2	Transformers	15
2.5.3	Hybrid architectures	16
3	MATERIALS AND METHODS	18
3.1	Datasets	18
3.2	Data loading and transformations	19
3.3	Model architecture	22
3.4	Self-supervised pre-training algorithms	23
3.4.1	MIM approach	23
3.4.2	BYOL approach	24
3.5	Supervised and self-supervised training	25
3.5.1	Supervised training	27
3.5.2	Self-supervised training	28
3.6	Testing	30
4	RESULTS	31
4.1	Supervised training results	31
4.2	MIM pre-training results	34
4.3	BYOL pre-training results	35
4.4	MIM fine-tuning results	36

4.5	BYOL fine-tuning results	39
4.6	Testing results	41
5	DISCUSSION.....	43

LIST OF FIGURES

FIGURE 1 Self-supervised learning pipeline.....	4
FIGURE 2 Diagram highlighting the difference between self-supervised learning (a) and transfer learning (b) [17].....	5
FIGURE 3 Contrastive learning diagram.....	6
FIGURE 4 NuMorph overview of tissue processing, imaging, and image analysis procedures [3].....	12
FIGURE 5 U-Net architecture for a 572x572 input with one channel [75].	15
FIGURE 6 (a) The architecture of a Swin Transformer; (b) two successive Swin Transformer blocks. W-MSA and SW-MSA are multi-head self-attention modules with regular and shifted windowing configurations, respectively [79].	16
FIGURE 7 UNETR architecture. A 3D input with an arbitrary number of channels (4 in this example) is divided into non-overlapping patches of resolution P and linearly embedded into a space of size K . The sequence is added with a position embedding and used as an input to a transformer model. The encoded representations of different layers in the transformer are extracted and merged with a decoder via skip connections to predict the final segmentation. Here, $P = 16$ and $K = 768$ [83].	17
FIGURE 8 Middle slice example of a BraTS flair modality and a Selma cell nucleus volume in their original state, after base transforms (normalization and channel-first ordering), and the two augmented views obtained for BYOL. Scale bars are added for reference to the original size (first two panels) and the cropped size (last two panels), for a ROI of [120, 120, 96].....	21
FIGURE 9 SwinUNETR architecture with a BraTS input [84].	22
FIGURE 10 MIM schematic diagram showing the encoding, masking and reconstruction pipeline.	24
FIGURE 11 BYOL's architecture. BYOL minimizes a similarity loss between $q\theta(z\theta)$ and $sg(z\xi')$, where θ are the trained weights, ξ are an exponential moving average of θ and sg means stop-gradient. At the end of training, everything but $f\theta$ is discarded, and $y\theta$ is used as the image representation [23].	25
FIGURE 12 Supervised training from scratch for BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using the full annotated data splits. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset.....	31
FIGURE 13 Supervised training from scratch for Selma data. The graph on the left shows the training and validation losses for the Selma dataset using the full annotated data splits. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset.....	32

FIGURE 14 Supervised training from scratch for a reduced BraTS dataset. The graph on the left shows the training and validation losses using 10% of the annotated training data split. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints using the full validation split. 33

FIGURE 15 Supervised training from scratch for a reduced Selma dataset. The graph on the left shows the training and validation losses using 50% of the annotated training data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints using the full validation split. 33

FIGURE 16 MIM pre-training losses across 100 epochs for BraTS (graph above) and Selma (graph below)..... 34

FIGURE 17 This figure shows the middle slices of a processed cropped sample given as input to the model, the same sample with the mask applied at voxel-level, and the reconstruction obtained by the model. The panels above show the visualizations for a BraTS sample, and the panels below show the visualizations for a Selma sample. The mask applied in both cases covers 75% of the sample, and scale bars are added for clarity. 35

FIGURE 18 BYOL pre-training losses across 100 epochs for BraTS (graph above) and 50 epochs for Selma (graph below). 36

FIGURE 19 Fine-tuning of the MIM pre-trained model for full BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using the full training data. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset. 37

FIGURE 20 Fine-tuning of the MIM pre-trained model for reduced BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using 10% of the training split. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset. 37

FIGURE 21 Fine-tuning of the MIM pre-trained model for full Selma data. The graph on the left shows the training and validation losses for the Selma dataset using the full annotated data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset..... 38

FIGURE 22 Fine-tuning of the MIM pre-trained model for reduced Selma data. The graph on the left shows the training and validation losses for the Selma dataset using 50% of the training data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset. 38

FIGURE 23 Fine-tuning of the BYOL pre-trained model for full BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using the full training data. The graph on the right shows the Dice scores for each segmentation class,

the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset. 39

FIGURE 24 Fine-tuning of the BYOL pre-trained model for reduced BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using 10% of the training split. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset. 40

FIGURE 25 Fine-tuning of the BYOL pre-trained model for full Selma data. The graph on the left shows the training and validation losses for the Selma dataset using the full annotated data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset..... 40

FIGURE 26 Fine-tuning of the BYOL pre-trained model for reduced Selma data. The graph on the left shows the training and validation losses for the Selma dataset using 50% of the training data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset. 41

LIST OF TABLES

TABLE 1 Selma3D data information table [5].	19
TABLE 2 Training hyperparameters.....	26
TABLE 3 Testing results for the different trained Selma models. The Dice scores for each entity are shown, as well as the average Dice and test loss for all samples. Best performing models in each metric are marked in bold.	42
TABLE 4 Testing results for the different trained BraTS models. The Dice scores for each class are shown, as well as the average Dice and test loss for all samples. Best performing models in each metric are marked in bold.	42

ABBREVIATIONS

AI	artificial intelligence
BFM	bright field microscopy
BYOL	bootstrap your own latent
CNN	convolutional neural network
CT	computed tomography
EMA	exponential moving average
FAIR	findable, accessible, interoperable, reusable
FN	false negative
FP	false positive
LN	layer normalization
LSFM	light-sheet fluorescent microscopy
MAE	masked autoencoders
MIFA	metadata, incentives, formats and accessibility
MIM	masked image modelling
MLP	multilayer perceptron
MRI	magnetic resonance imaging
MSE	mean squared error
NGFF	next-generation file formats
OME	open microscopy environment
REMBI	recommended metadata for biological images
ROI	region of interest
SSL	self-supervised learning
SOTA	state-of-the-art
SW-MSA	shifted window multi-head self-attention
TP	true positive
ViT	Vision Transformer
W-MSA	window multi-head self-attention
WSL	weakly supervised learning

1 INTRODUCTION

Biomedical imaging plays a critical role in modern healthcare, providing useful data for diagnosis of diseases, assessment of biological processes, monitoring of certain conditions and much more. Precise and safe imaging macroscopy techniques like magnetic resonance imaging (MRI) and computed tomography (CT) have been key to detecting diseases such as cancer in early stages, which is essential for increasing survivability of the patient [1].

Microscopy modalities, while often overlooked, also play a critical role in the diagnosis of multiple diseases and in the assessment of biological processes. The field of microscopic imaging has evolved with time, overcoming limitations in penetration depth and managing to develop techniques that can image large 3D volumes at cellular and subcellular resolution [2]. One of these techniques is light-sheet microscopy, which has become increasingly popular due to its capability of imaging whole organs at incredible speeds, which enables the study of certain structures and pathways that would not be properly captured with other technologies [3].

Analyzing bioimages is a complex and time-consuming task, which causes biomedical research to advance slowly. With the creation of artificial intelligence and machine learning models, especially deep neural networks, images of all kinds could be processed much faster, providing a great aid for scientists. However, these models also have limitations, like the need for great amounts of annotated data to achieve good results; data that is hard to obtain in specialized fields such as biomedicine. Self-supervised learning emerged to overcome the need for labels, matching or even surpassing the performance of regular supervised learning models while using a fraction of the annotated data [1].

However, due to the wide range of self-supervised techniques created and the variety of bioimage modalities that exist, each with their own purpose and qualities, it is hard to obtain a clear understanding of the current landscape of self-supervision for applications involving bioimaging. Furthermore, most research usually revolves around the most popular and well-studied modalities due to their availability of benchmarks and data, creating research gaps for emerging but useful technologies like light-sheet microscopy.

This thesis explores how self-supervised learning impacts the performance of a deep learning model in the task of segmenting a 3D light-sheet microscopy volume, with the goal of providing a clearer understanding of how the used data, model and self-supervision techniques work with each other, and what factors influence most in the results. The value of this research lies in the fact that no detailed, comparative analysis of self-supervision techniques has been done for light-sheet microscopy; the experiments performed aim to shed light on this obscure area.

For this purpose, a complete custom self-supervised training pipeline is developed, containing modules for data loading, data augmentation and preprocessing, supervised training and fine-tuning, pre-training with two different self-supervision techniques, model architecture initialization and customization, testing and result plotting. The pipeline is tested on the BraTS [4] and Selma3D [5] datasets; the first is chosen due to it

being a popular MRI data benchmark, which means there are multiple studies using it for segmentation tasks, providing a baseline to compare the pipeline performance with and check if the different modules work as expected. The Selma3D dataset is selected for being one of the only publicly available light-sheet microscopy datasets which contain both unannotated and annotated data. Below is an overview of how this thesis is structured for clarity.

First, a background check of the current literature regarding self-supervised learning is done to understand what methods exist, what benefits they have with respect to standard supervised learning and what architectures they work best with. The bioimage modalities most commonly used in biomedical research are also reviewed to get a clearer grasp of the properties of the data that will be handled during the experiments, with a focus on the specific modalities used; MRI and light-sheet microscopy. The existing studies researching self-supervised learning in bioimage applications are also reviewed to learn what architectures and techniques have been found to work best with different bioimage modalities.

Then, the materials and methods section describes in depth the properties of the selected datasets, how the data is structured and prepared for each training approach, what data augmentations are applied to the data and why, architecture details of the selected model, how the pre-training methods work and their components, and how the training and testing pipelines function. The selected pre-training approaches for this thesis are masked image modelling and BYOL, and the architecture used is SwinUNETR. The reasons behind choosing them are explained in the background and methods sections.

The results section contains the output obtained from the experiments done, which compare the performance of the model for each dataset and each training approach, as well as providing the losses and metrics obtained. The results are reviewed in the discussion section, where an explanation for why the results obtained are as they are is given, as well as the conclusions that can be gained from them. At the end of the document, an appendix is added which contains the declaration for AI usage and the hardware and software specifications for the code.

2 BACKGROUND

To understand the work contained in this thesis, it is first necessary to review the concept of self-supervised learning (SSL): how this method works, the different types of techniques that it uses, and the advantages and disadvantages with respect to other machine learning approaches. This project focuses on the application of SSL in 3D bioimage segmentation, so the state-of-the-art (SOTA) in this field will also be explored, both regarding the best performing techniques and the most innovative architectures. The different types of bioimages and their qualities will also be explored to better understand the datasets used.

2.1 What is self-supervised machine learning?

Machine learning algorithms have evolved rapidly throughout the years, reaching performances that surpass those of humans on complex tasks such as object recognition and semantic segmentation [6]. This success was initially mostly driven by supervised learning, a technique where the model must learn to map input variables (data points) to output variables (labels) by using data manually labeled by humans as training samples [7]. However, this approach has important limitations; the most evident one being a heavy reliance on high quality annotated data, which is expensive to obtain. It is also vulnerable to adversarial attacks, spurious correlations, and generalization errors [8].

Unsupervised learning overcomes the need for manual labeling by learning directly from the intrinsic features contained in the unannotated data [7]. This approach aims to discover patterns among the samples for tasks such as clustering, dimensionality reduction and anomaly detection [8]. Nevertheless, classic unsupervised learning methods like principal component analysis or k-means clustering also have their own limitations; they struggle with complex downstream tasks due to the learned representations being too general or ambiguous, which is caused by the lack of a precise ground truth given by labels [9].

SSL is usually considered a type of unsupervised learning, although it is more easily understood as its own type. In this approach, the model is first trained using unlabeled data and is then adapted to a specific task using labeled data. The motivation behind the development of SSL is to leverage the vast amount of unlabeled data available to learn useful representations like in unsupervised learning and then use a small amount of labeled data to obtain the high accuracy interpretable results typical of supervised learning [10].

SSL learning has two phases: the pretext tasks (or pre-training), and the downstream tasks (or fine-tuning). During pre-training, the model only needs unannotated data, from which artificial supervisory signals are generated. These signals are used as the target to learn the features needed for completing the tasks. Once this phase is completed, the weights are preserved for the fine-tuning phase, which requires a small amount of annotated data. During this phase, the model learns to associate the features learnt from the unannotated data to the labels now provided [11].

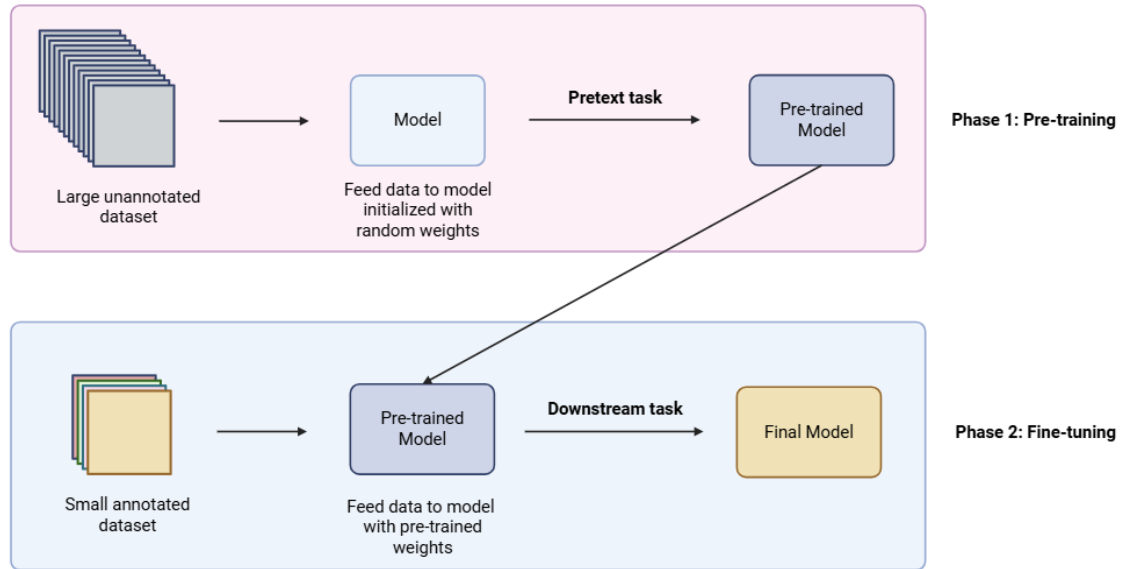


FIGURE 1 Self-supervised learning pipeline.

It is important to make the distinction between SSL and weakly supervised learning (WSL), as both use a combination of unannotated and annotated data in their workflow but have different methodologies. As said previously, SSL consists of two phases, in which each type of data is used separately. In WSL, both labeled and unlabeled data are used simultaneously during training [12].

The following techniques belong to WSL, they must not be confused with SSL: incomplete supervision, inexact supervision and inaccurate supervision. In incomplete supervision, a small amount of fine-grained labeled samples is mixed with a great number of samples without label, so the model can use the additional information given by each of the data types to improve its understanding about the other [13]. In inexact supervision, all samples are annotated with coarse-grained labels, so the model must learn to extract the finer features from the high-level information given by the data. The last technique is inaccurate supervision, where some of the labels are incorrect to imitate mistakes during the annotation process. The model must then learn to extract useful and accurate information even when some of the samples are not ground truth [14], [15].

Similarly, transfer learning has many similarities to SSL, as both pre-train a model using a larger dataset and then adapt the model to a downstream task using a smaller task-specific dataset. However, transfer learning uses a supervised learning approach with annotated data for both pre-training and fine-tuning, as opposed to SSL that uses unannotated data with custom pretext tasks for pre-training [16].

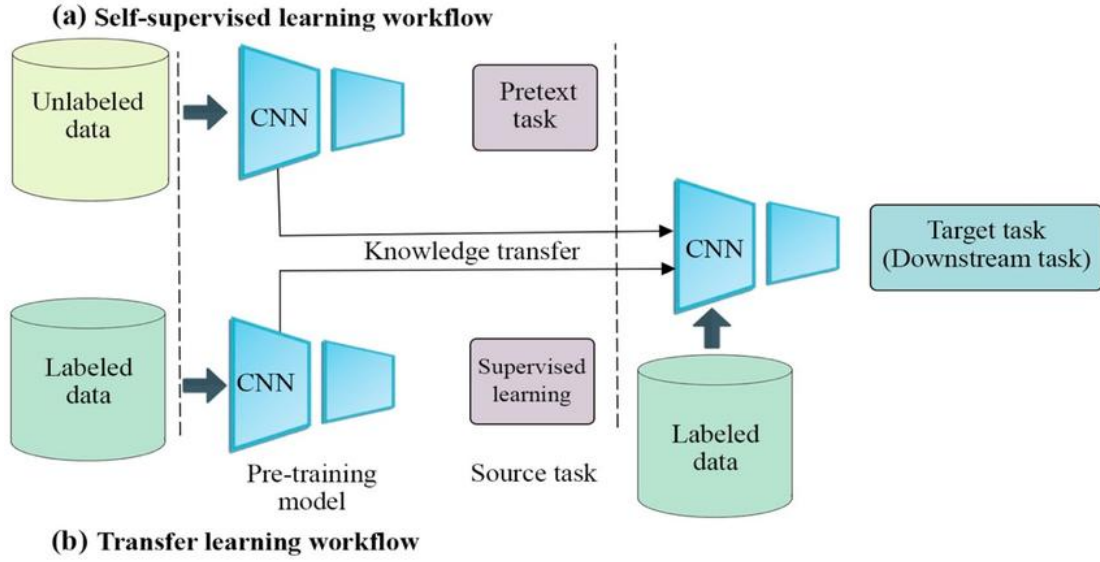


FIGURE 2 Diagram highlighting the difference between self-supervised learning (a) and transfer learning (b) [17].

The self-supervised approach has multiple advantages with respect to a purely supervised or unsupervised approach. It usually learns better representations and has a higher performance on the downstream task than purely unsupervised training, and it requires much less labeled data than the supervised approach while matching or even surpassing its results [10]. This is relevant because annotating data is time consuming and costly, as it usually requires experts to make sure the labels are of high quality. Unlabeled data is much easier to obtain, so models that leverage it are more time and cost effective. In addition, it has been shown that self-supervised models are more generalizable than their supervised and unsupervised counterparts, achieving better transferability and a reduced performance gap between the general task and the specific task [11].

However, the SSL approach also has some limitations: it is usually more computationally expensive than other methods due to the requirement of large model sizes, plenty of data, and more iterations to achieve SOTA performances. This can turn out to be prohibitively expensive for individuals without the necessary resources, limiting its usability. The SSL pipeline is also relatively complex compared to end-to-end approaches, as it involves multiple steps and requires customization of the pretext task and data augmentations to reach a good performance in the downstream task [18], [19].

2.2 Types of self-supervised learning

The learning techniques for the self-supervised approach are varied and can be roughly divided into three categories: contrastive, non-contrastive and generative learning.

2.2.1 Contrastive learning

Contrastive learning consists of creating augmentations from existing data, and making the model pull the representations that come from the same sample (positive pairs) closer together in the feature space, while separating representations that come from different samples (negative pairs). This is done using a contrastive loss like InfoNCE or NCE,

which contain a repulsion term to push negative samples apart [17]. Two well-known frameworks belong to this category: MoCo and SimCLR.

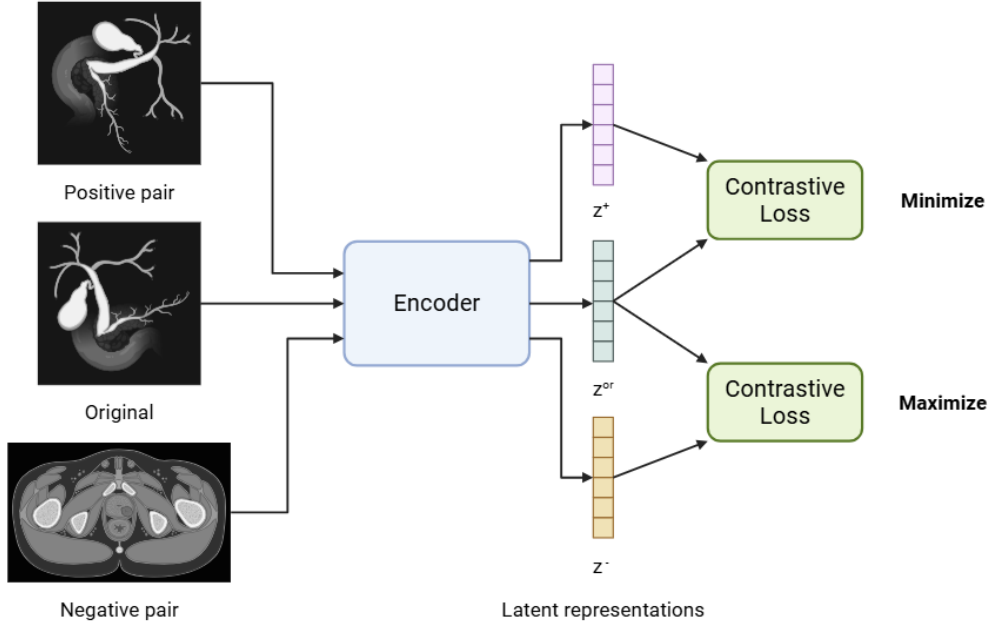


FIGURE 3 Contrastive learning diagram.

MoCo consists of two encoders, query and key, in which the first is updated by backpropagation and the second is updated by exponential moving average of the former. This framework contains the negative samples generated by the momentum (key) encoder in a dictionary, and when a query is passed, it is compared to the keys, giving a low contrastive loss when the query is similar to its positive key and dissimilar to the rest of negative keys [20]. MoCo needs smaller batch sizes than SimCLR and keeps stable negative representations thanks to the momentum updater, but it has a more complex pipeline than SimCLR [17].

SimCLR uses a single encoder combined with a projection head, updated by backpropagation. The model takes a mini batch of instances and performs augmentations on them. Then, one instance and its augmentation are selected as the positive pair and the rest are treated as negative examples. SimCLR confirmed that, for contrastive learning, only augmentations and contrastive loss is enough, making it a simple technique. However, it requires very large batch sizes, and its performance depends heavily on the data augmentations chosen [21].

2.2.2 Non-contrastive learning

Non-contrastive learning is similar to the former in that it compares representations of augmented views, but it does not need negative samples or a contrastive loss to learn. Instead, it uses only positive samples and avoids collapse by using architectural asymmetry in the model [10]. The most relevant frameworks that use this technique are Bootstrap Your Own Latent (BYOL), SimSiam and DINO.

BYOL and DINO use the self-distillation method to avoid collapse: a teacher network supervises the student network and uses its own predictions [22]. BYOL uses two deep

neural networks with the same architecture but different weights (Siamese networks). One network acts as the target network, which encodes and projects the input to a latent representation and is updated using exponential moving average (EMA). The other acts as the online network, which tries to predict the target's representation and is updated using backpropagation. BYOL was designed specifically for self-supervised image learning, matches or exceeds contrastive methods and is better matched for small batch sizes [23].

DINO builds upon BYOL using also a student network that tries to predict the teacher's output and updating the teacher network using EMA. However, instead of matching representations it matches probability distributions, uses temperature sharpening and centering instead of a stop-gradient as the stability mechanism, and produces more clustered and semantic representations [24].

SimSiam is also non-contrastive but is not a self-distillation method like the former ones. It creates two augmented views of a sample, processes them through an encoder and MLP projection head, and tries to reduce the negative cosine similarity between the resulting representations. It is simpler and more efficient than BYOL and DINO as it does not use momentum encoders and both identical networks share weights, but it is less stable than these methods [25].

2.2.3 Generative learning

Generative learning consists of training both an encoder to encode the input into an explicit vector z , and a decoder to reconstruct x from z . Among this technique, the most popular approach is the autoencoding model due to its flexibility and good performance. Its goal is to reconstruct missing or corrupted parts of the input, following the idea that if the model is capable of doing so then it must understand the data's meaning and structure [8].

Masked image modelling (MIM) is a generative learning technique that has managed to challenge the dominance of contrastive learning in SSL. In MIM, the input image is divided into patches and partially masked, and the model must reconstruct the missing parts using the visible context. The well-known method of masked autoencoders (MAE) belongs to this category, as well as others like BEiT and SimMIM [11].

MAE reconstructs the raw pixels of masked patches using an autoencoder; the encoder only processes the visible patches, and the decoder reconstructs the masked tokens. The pre-trained encoder is kept for the fine-tuning phase and the decoder is discarded. The loss used for MAE training is usually the mean squared error (MSE) between the reconstructed and the original patches [26]. This method makes the model learn global context, scales efficiently to large images or volumes, performs well with dense tasks like segmentation and is stable. However, it requires optimizing the loss and hyperparameters like the masking ratio to obtain good results and avoid the domination of low-level details in the reconstruction [27].

SimMIM is a simplified MAE: the encoder input is the full image with mask tokens, the loss is calculated between pixels instead of patches and usually it contains a shallow

prediction head instead of a proper decoder. It is less efficient than MAE at large scale, but it is simple, requires less architectural complexity and has good empirical performance [28].

BEiT is also similar to MAE but instead of using the raw pixels as targets, it uses the token output of a tokenizer. This tokenizer is pre-trained to convert the input images into visual tokens called codebook indices, and the model must then predict the codebook index of the masked patches. This turns the pretext task into a classification problem with a reconstruction goal, and as such it uses cross-entropy loss between the predicted and the true codebook index to learn. This method encourages the model to learn high-level semantic features instead of low-level texture, but it requires a tokenizer which is complex and expensive to train [29].

2.3 Bioimage modalities across biological scale

Bioimage refers to any image acquired from biological systems at cellular, tissue or organ level using imaging modalities such as microscopy, CT, MRI, PET and ultrasound. They are widely used for research and play an important role in machine learning applied to biomedical tasks, due to the models needing many task-related training samples to achieve good performance and generalization. Recently, there have been efforts to create open collections of diverse high-quality bioimages such as the BioImage Archive [30]; this is a highly valuable contribution as it allows the public to access data that can otherwise only be obtained through expensive imaging technologies and biomedical expertise.

Bioimages used for research are usually subjected to certain standards that ensure their quality and the reproducibility of the studies they are used in; the main ones are FAIR [31], OME-NGFF [32], REMBI [33] and MIFA [34], which regulate the properties and formats that bioimages and their metadata should fulfill to be added to these archives. FAIR establishes that data needs to be findable, accessible, interoperable and reusable; for this, unique and indexed identifiers for the data which are retrievable by an open communications protocol should be used, and metadata should be clear and rich with a formal language [31]. OME-NGFF introduces a specific, cloud-friendly, hierarchical data format that allows users to store and rapidly access large multidimensional images [32]; it is part of OME [35], a framework that develops open-source tools and standards for bioimages ensuring interoperability between different modalities and software.

REMBI describes an 8-component framework of all the information that should be included in a bioimage's metadata to make it reusable: study, study component, biosample, specimen, image acquisition, image data, image correlation and analyzed data are the modules that REMBI defines. MIFA focuses on making annotated data standards to ensure reusability for AI applications; the annotation file should be clearly linked to the original image and its metadata, which should fulfill the appropriate standards themselves, the annotations should contain a standardized metadata file with a cloud-compatible format, and credits should be given to the creators of the annotations to incentivize data production.

These standards were originally created for microscopy data, as most bioimages were obtained from histological research. Macroscopy modalities such as CT or MRI are mostly used in clinical imaging and usually have patient-identifiable information that makes them unfit for bioimage archives. Eventually, macroscopy datasets created specifically for research appeared, which complied with the aforementioned standards and allowed them to be added to these archives [30].

2.3.1 Medical macroscopy modalities

Macroscopy imaging refers to techniques that capture visualizations of structures that are visible to the naked eye, usually on a scale larger than one millimeter [36]. Macroscopy is widely used in clinical imaging due to its capability to visualize large-scale structures like organs or even the full body while the organism is alive in a non-invasive way, causing little to no harm. The main macroscopy modalities used in biomedical imaging are computed tomography (CT), magnetic resonance imaging (MRI), X-rays, nuclear medicine and ultrasound [37]. This section will focus on CT, MRI and nuclear medicine as those are the modalities most represented in deep learning studies for biomedical applications [38].

CT obtains detailed, cross-sectional, 3D images of soft tissues, blood vessels and bones by using a rotating beam of X-rays that takes hundreds of snapshots from different angles. These snapshots are then passed to a computer which reconstructs the full slices and volumes using mathematical algorithms. The advantage of CT lies in its speed and usability, as it only takes minutes to obtain the final scan, while being very detailed and allowing to visualize multiple biological structures at once. However, it uses radiation which can be harmful if used extensively, so it is mostly used in case of emergencies or when other imaging techniques have not been enough to diagnose the issue [39]. The BTCV dataset, which is widely used for organ segmentation tasks, consists of multi-organ CT images with annotations [40].

MRI generates high-resolution 2D and 3D images of organs and soft tissues by using magnets and radio waves to excite water molecules in the body. It is non-invasive and causes no harm to the body as it does not use ionizing radiation. MRI has become the preferred modality for imaging the brain, spine, and pelvic organs as it can achieve greater contrast between soft tissues than other modalities while causing no harm. The disadvantages of this technique are that it takes long to obtain the final scan, sometimes more than an hour, and that it cannot be used on patients with metal implants due to the magnetic fields being used [37]. The BraTS dataset consists of multi-modal MRI brain tumor images with annotations, also widely used for training AI models for organ segmentation [4], [41].

Nuclear medicine uses safe, small amounts of radioactive material injected into the body to image organ function and metabolic processes, by using special cameras that detect the gamma radiation emitted from the inserted substance. Instead of imaging biological structures, it focuses on visualizing functional processes in the body even at cellular level, which allows it to detect diseases in earlier stages compared to other modalities. However, it has a higher radiation dose and lower resolution than other methods, it can take up to

several hours until the radiotracer is distributed in the body and is not recommended for certain patients such as those undergoing a pregnancy [42].

2.3.2 Microscopy modalities in biology

Microscopy in the context of biology refers to imaging techniques that are capable of visualizing structures at cellular and subcellular resolution; it is mainly used for histology research as it can provide detailed visualizations of tissue, cells and molecules from biological samples. One of the fundamental modalities is bright field microscopy (BFM), which consists of illuminating a sample with white light to create contrast with a bright background. This allows to observe both fixed and live thin samples, although it usually requires staining to increase contrast, which in most cases involves killing the observed specimen. BFM is widely used for histological studies due to its cheapness and simplicity, with hematoxylin and eosin-stained slides being one of the most common microscopic images available for a variety of tissues and cells [43].

New microscopy techniques emerged to overcome BFM's limitations on resolution, visualization depth and contrast, some even allowing the creation of small 3D image stacks; the most popular ones are fluorescence and confocal microscopy. Fluorescence microscopy labels molecules of a specimen using fluorescent probes called fluorophores, which when excited with high energy light, release visible light that can be observed. This technique achieves high contrast and resolution due to having a variety of differently colored fluorophores that tag specific molecules, allowing each marked structure to be easily distinguished from others. Fluorescence microscopy also enables the imaging of live cells and biological processes, as fluorophores can be introduced in a live sample [2], [44].

Confocal microscopy is a technique that increases visualization depth by using a focused laser to illuminate a specific spot and a pinhole to block light that is not from the desired focal plane, allowing to image live deep tissue with high contrast and resolution. 2D planes are scanned using a raster pattern, these planes can be stacked to achieve a 3D image. However, this technique is time consuming, expensive, complex, can damage the tissue over time due to the intense laser light and is still relatively limited in depth, reaching its limit at ~100 micrometers, compared to the ~10 micrometer limit of BFM [2], [44].

The next challenges to tackle were achieving sub nanometer resolution and improving the quality of microscopic 3D images. Electron microscopy managed to surpass the usual 200 nanometer resolution limit of light microscopy modalities by using a beam of electrons in a vacuum instead of photons to illuminate the sample. As electrons have a much shorter wavelength, they can achieve magnifications of one or more million times, allowing the detailed visualization of atomic structures, organelles, viruses and molecular complexes. There are multiple sub modalities of electron microscopy: scanning electron microscopy scans sample surfaces to create 3D images, transmission electron microscopy shoots the electrons through a very thin sample to visualize internal 2D structure, and cryo-electron microscopy allows to image samples in a near-native state by flash freezing them [45].

As explained, some of the previously mentioned modalities are capable of imaging microscopic-resolution volumes, however the main limitations point-scanning techniques face are significantly lower resolution in the depth-axis compared to the 2D axis in uni-axial microscopes, slow-speed which difficulties imaging of live processes, and higher photobleaching and phototoxicity due to the prolonged light exposure needed for 3D scanning [46].

Light-sheet fluorescence microscopy (LSFM) overcomes these limitations by illuminating an entire plane section at a time, reducing significantly photobleaching and imaging time when compared to point-scanning techniques like two-photon or spinning disk confocal microscopy, which were previously the SOTA for in vivo 3D microscopic imaging due to their relatively good penetration depth and resolution. Because of these advantages, LSFM has become SOTA for 4D imaging of large, transparent specimens like embryos, organoids and cleared tissues. However, it has some limitations, such as having less axial range and lower resolution than other 3D techniques, as well as being non-trivial to implement [46], [47].

An important step for microscopy research was the transition from analog to digital imaging, also known as whole slide imaging (WSI) or virtual microscopy. WSI consists of scanning glass pathology slides at high resolution to create digital images with multiple magnification views, allowing remote analysis, easy sharing and safe keeping of microscopy data. This technique can be used for multiple microscopy modalities and even for 3D imaging, as multiple 2D slices can be stacked using computational methods. WSI is especially relevant for machine learning applications, as it provides large amounts of data for analysis and training [48].

However, most WSI data comes from point-scanning microscopy techniques or traditional BFM histological slices, especially when it comes to annotated images. Volumetric microscopy data from large samples such as whole organs is relatively scarce, with modern techniques like LSFM being very underrepresented. That is why tools like NuMorph, which is specifically designed to aid annotation and analysis of whole brain LSFM volumes, are so important, as they facilitate the creation of high-quality data from underrepresented modalities [3].

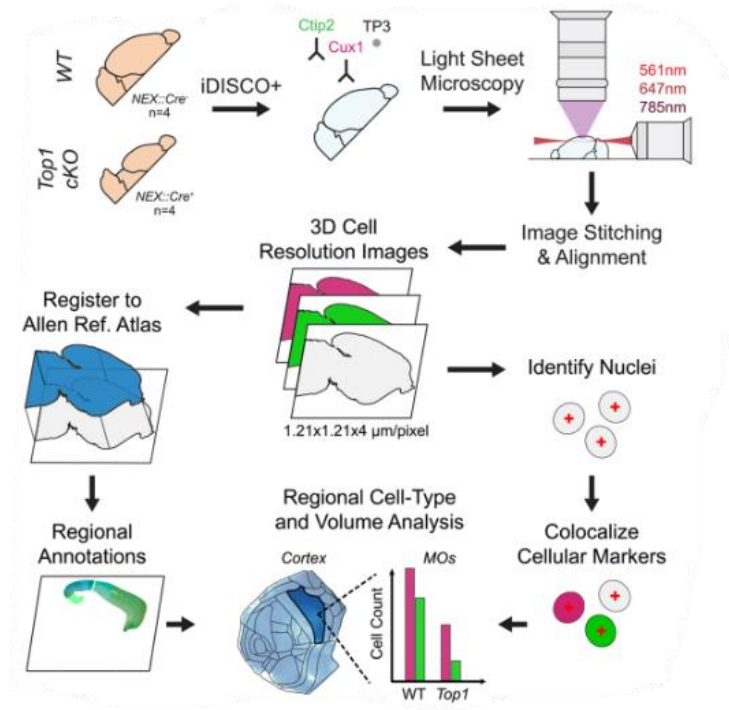


FIGURE 4 NuMorph overview of tissue processing, imaging, and image analysis procedures [3].

2.4 Self-supervised learning in bioimage applications

SSL is particularly relevant for medical applications, as most medical AI relies on supervised approaches, which as said previously require a large amount of annotated data. In medical contexts, labeled data is even harder to obtain, as the labeling must be done by highly specialized people and it must be of very good quality due to the complexity and importance of the tasks, furthermore it takes significantly more time to label medical data than natural data [49].

In the field of AI applied to bioimages, a solution proposed for this issue was to take a model pre-trained on an already existing large dataset of annotated natural images like ImageNet, and then re-train them on a smaller task-specific medical dataset. This approach obtained better performance than training the model from scratch, however it is not optimal because the transfer learning benefit depends on how similar the pre-train dataset is to the medical dataset, which in many cases is not much [50].

SSL tackles both issues by needing significantly less labeled data and also having a medically relevant pre-training by leveraging the large quantities of unannotated data in the medical field. This approach has been used for multiple applications like image segmentation and classification, gene or protein sequence analysis, diagnosis based on electronic health records and many more, reaching or even surpassing supervised learning performance [49].

For tasks involving bioimages, contrastive methods such as SimCLR and MoCo have been the most popular choice for self-supervised pre-training. However, with the development of new architectures and techniques, other methods like DINO, BYOL, and MIM have demonstrated comparable or even improved performance in tasks such as segmentation and classification. The choice for which method to use depends heavily on

the dataset size, imaging modality, the downstream task, and the model architecture, as no method is universally better than others and each excels in different contexts [51], [52].

Generally, research shows that contrastive and non-contrastive methods excel in classification tasks for both large-scale dense modalities like CT and MRI, and small-scale modalities like microscopy. This is due to their focus on learning high-level representations, texture patterns and global semantic features over pixel-wise details. They also perform well in segmentation tasks, with methods like DINO, BYOL, SimCLR and MoCo being among the techniques with highest performance [52], [53].

In contrast, generative methods are usually not as strong in classification tasks when compared to other methods, due to their focus on learning fine-grained details instead of the high-level differences between features of different entities [53]. Where generative methods excel is in segmentation tasks, as MIM techniques have been shown to outperform contrastive and non-contrastive methods in multiple modalities like CT, MRI and even microscopy. They excel specially in tasks with small and volumetric datasets, where the performance gap with respect to contrastive and non-contrastive methods is bigger [51], [54].

It is also important to adapt the self-supervision methods to the dimensionality of the data. Initially, when the downstream tasks involved classification or segmentation of 3D bioimages, the most common approach was to divide the volume into 2D slices and train the model on these. This had some advantages, like artificially increasing the dataset size and reducing computation costs, but also the disadvantage of losing the full spatial context, which is especially relevant for anatomical structures. Therefore, to achieve the highest performance in 3D bioimage tasks, it is beneficial to design self-supervised pre-training tasks that work with the full 3D volume [55].

Most research evaluating SSL methods in medical imaging analysis focuses on the modalities of CT and MRI [56], [57], [58], [59], [60], [61], with X-rays also being relatively well-represented [62], [63], [64], [65]. These studies cover a wide range of organs, including lungs, liver, kidneys, brain, heart, and prostate, and various applications such as image registration, synthesis, false-positive reduction, classification and segmentation, with the last two accounting for the majority of studied tasks.

There is also some research about the applications of SSL in histology [66], [67], [68], which focuses mostly on classification and segmentation of microscopic organ structures, cells, diverse tumors and other histopathology elements. The least studied modality is the ultrasound [69], [70], with just a few studies applying already existing architectures and techniques to try and improve performance on classification tasks.

Regarding SSL applications in microscopy, it is important to highlight that most research in this area focuses on the modalities most commonly used for histopathology, which are electron microscopy and optical microscopy. More specifically, scanning electron, confocal, fluorescent and brightfield microscopy constitute the majority of studies [71]. In comparison, the application of SSL in more modern microscopy techniques remains

relatively underexplored; light-sheet microscopy in particular is a modality that is gaining popularity but only has a couple studies analyzing SSL applications on it [5], [72].

SSL in light-sheet microscopy is mainly explored in the Selma3D challenge, which aims to gain insights on how self-supervised training can improve model performance and generalization in microscopy segmentation, specifically in large 3D volumes of whole organs. For this, the challenge organizers provided an extensive dataset containing both unannotated and annotated brain volumes with diverse biological structures. The participants submitted solutions consisting of a variety of backbones, SSL strategies, data preprocessing and fine-tuning approaches, obtaining a better performance than the fully supervised baseline. However, due to each team submitting their own custom solution and achieving different results, it is difficult to conclude which design choices have the most influence in the final performance [5].

2.5 Popular deep-learning architectures for bioimage analysis

For optimal performance in machine learning, it is not only necessary to choose an appropriate training algorithm, but also to select a model backbone that works well with both the data and the learning method. In this section, the most popular architectures for image analysis will be explored, highlighting the models that have become SOTA in bioimage segmentation.

2.5.1 Convolutional neural networks

Computer vision was dominated by convolutional neural networks (CNNs) for a long time, due to their efficiency and great capability to learn local and global spatial features. ResNet-like architectures [73] were SOTA in large-scale image recognition: residual blocks learnt the difference between the input and desired output instead of direct mappings, and skip connections allowed inputs to bypass layers and be added directly to deeper-layer outputs. These innovations allowed models to overcome the vanishing gradient issue and allow deeper architectures for more complex learning, becoming a staple in both natural and medical imaging tasks [74].

Later studies tried to optimize CNNs for biomedical tasks, which usually require detailed segmentation while using a relatively small dataset compared to natural image tasks. One architecture in particular, the U-Net, achieved great results while keeping the model efficient, becoming SOTA for biomedical segmentation. This model consists of a symmetrical encoder-decoder architecture; the encoder repeatedly downsamples the input until reaching a bottleneck, where the final high-level representation is passed to the decoder. The decoder then repeatedly upsamples this representation, reconstructing the input image with the help of skip connections from the encoder levels. Finally, a simple segmentation head maps the feature vector to the segmentation output [75].

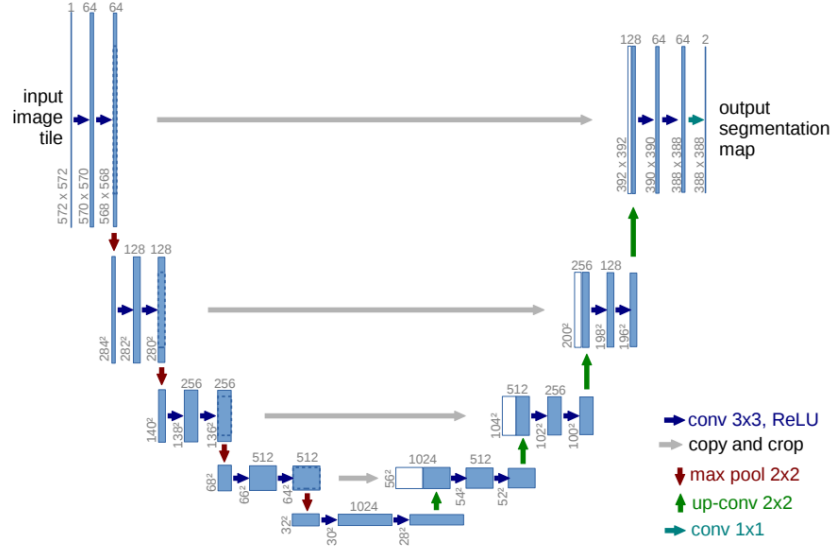


FIGURE 5 U-Net architecture for a 572x572 input with one channel [75].

2.5.2 Transformers

With self-attention-based architectures, especially Transformers, becoming SOTA in natural language processing [76], multiple studies trying to apply these models to the visual domain emerged. There were attempts to combine CNN-like architectures with self-attention mechanisms, and to drop convolutions altogether. However, the most successful approach was to apply the standard Transformer architecture directly to images with the fewest possible modifications; this led to the creation of Vision Transformers (ViTs) [77].

Instead of processing images pixel by pixel like CNNs, ViTs split the image into small pieces called patches, flatten them into 1D vectors, and project them into a smaller, fixed-size embedding space. Then, positional embeddings are added to retain information about each patch location in the image, as well as learnable classification tokens. Everything is passed through a Transformer encoder with self-attention, and then through an MLP head that gives the classification output. This architecture was shown to outperform CNNs in classification tasks while using substantially fewer computational resources for large-scale training [74].

Due to the success of ViTs [78], multiple studies looked to design a Transformer-based model that could be used as a general-purpose backbone, not only for classification but for dense vision tasks like segmentation too. This was achieved by the creation of the Swin Transformer, which addresses the exploding computational cost of training ViTs for high-resolution dense tasks by substituting global self-attention with self-attention computed within local shifted windows. This model was shown to outperform SOTA architectures in classification and segmentation tasks [79].

It first performs a patch partition and linear embedding on the input, which is then passed through several stages composed of two transformer blocks and a patch merging layer. The windowing strategies reduce the cost of self-attention from quadratic to linear relative to the image size, with shifting added to allow the model to learn how the windows

connect with each other. The merging layers allow the model to build a hierarchical, multi-scale feature map ideal for dense prediction tasks [79].

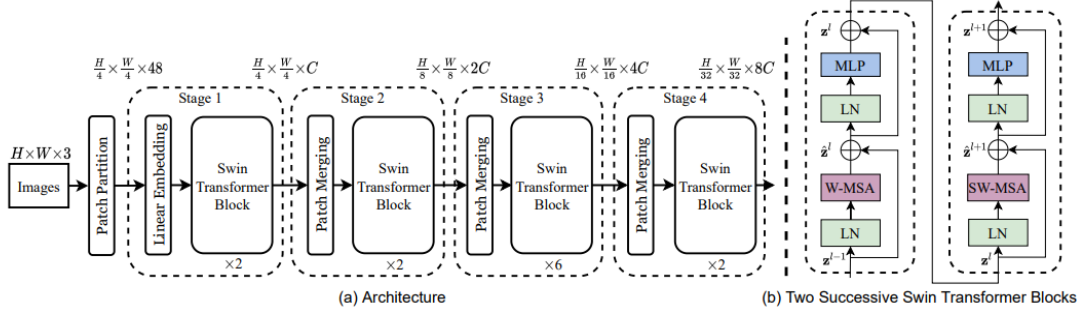


FIGURE 6 (a) The architecture of a Swin Transformer; (b) two successive Swin Transformer blocks. W-MSA and SW-MSA are multi-head self-attention modules with regular and shifted windowing configurations, respectively [79].

Following the success of the U-Net and the Swin Transformer, a new architecture emerged with the aim of leveraging the qualities of these two models, called the Swin-UNET. It takes the U-shaped encoder-decoder system with skip connections but replaces the convolutional feature extraction blocks with Swin Transformer blocks, which have windowed attention for computational efficiency. The encoder extracts local and global features hierarchically, and the symmetric decoder has patch expanding layers that upsample the representation to the original resolution. This architecture was tested on multi-organ and cardiac segmentation tasks, outperforming some full convolutional methods and architectures that used ViTs in combination with CNNs [80].

2.5.3 Hybrid architectures

The previous architectures each have their strengths, but all of them were initially designed to work with 2D natural images, with the exception of U-Net and Swin-UNET which were designed for 2D biomedical segmentation. Although pure CNN-based and Transformer-based architectures have been shown to also perform well in tasks involving bioimages, multiple studies show that hybrid architectures which mix CNN and Transformer qualities often achieve the highest performances when it comes to biomedical segmentation, both for 2D and 3D data. These models combine the strong spatial inductive bias and feature extraction capabilities of CNNs to obtain fine-grained local structures, while also using Transformers' self-attention to better capture long-range dependencies [81], [82].

UNETR is an example of a hybrid architecture specifically designed for 3D biomedical segmentation. This model is based on the U-Net, with the encoder being replaced by a stack of classic ViTs which are connected to the CNN decoder by skip connections and a bottleneck like the original U-Net. The input is taken directly as 3D, unlike other models that split the volume into 2D slices for processing; this allows the model to also learn depth information for better reconstructions. The input is divided into non-overlapping patches, flattened, projected to a 1D embedding space, and fed into the ViT stack. The final representation is upsampled by the CNN decoder to reconstruct the image, with the

help of the skip connections to merge the low-level, high-resolution details from CNNs with the high-level, global information from Transformers. This model outperformed other hybrid and pure CNN-based models on multi-organ 3D medical image segmentation [83].

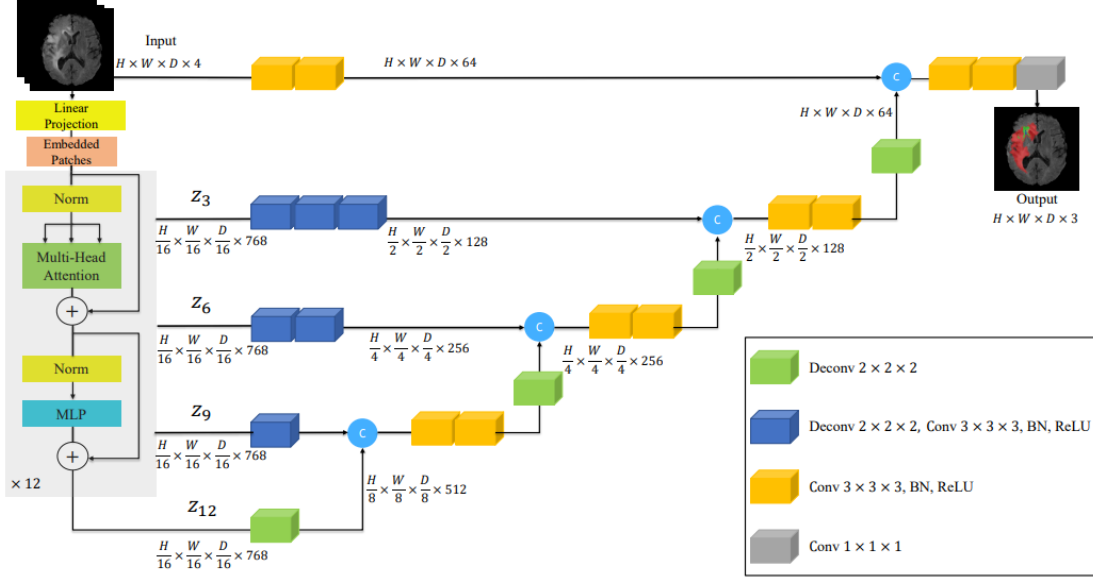


FIGURE 7 UNETR architecture. A 3D input with an arbitrary number of channels (4 in this example) is divided into non-overlapping patches of resolution P and linearly embedded into a space of size K . The sequence is added with a position embedding and used as an input to a transformer model. The encoded representations of different layers in the transformer are extracted and merged with a decoder via skip connections to predict the final segmentation. Here, $P = 16$ and $K = 768$ [83].

Another architecture that improved upon UNETR is SwinUNETR, which also leverages the strengths of U-Net and Transformers for 3D medical image segmentation, but instead of vanilla ViT blocks for the encoder it uses Swin Transformer blocks. These blocks extract features at multiple resolutions, using window-based self-attention for computational efficiency. The shifting of the windows allows the model to learn wider context between layers, not just local. The decoder is as in UNETR; a CNN that upsamples the features to reconstruct the original input, and which is connected to the encoder by skip connections that allow it to preserve fine-grained details. This model achieved SOTA results in 3D segmentation on BraTS (MRI brain tumors) and BTCV (multi-organ CT) [84].

There is research showing that there are certain combinations of architectures and self-supervision techniques which usually work better together compared to others. Contrastive methods tend to achieve higher performance when paired with CNN-based models, while generative methods have outperformed SOTA contrastive methods when paired with ViTs. However, this is based on general trends and there are exceptions; there have been successful attempts to adapt MAE so it can be used with CNNs, outperforming contrastive methods on certain benchmarks [51]. Non-contrastive methods like DINO and BYOL do not seem to have better synergy with a certain architecture over others, performing well with either type of backbone and being more influenced by the initialization strategy and downstream task [52].

3 MATERIALS AND METHODS

This section reviews the different tools, methods, programs and data that were used in this thesis. First, the selected datasets and their reason for being chosen are described, along with how the programming script created to load them in the proper way is structured. Then, the data augmentations and preprocessing transformations used to prepare the data for training are explained. The model architecture used, SwinUNETR, is also described component by component, along with the SSL algorithms chosen for the pre-training phase. Finally, the main scripts used for self-supervised and supervised training are reviewed, along with the testing script.

3.1 Datasets

The datasets used for this project belong to the BraTS 2021 Task 1 and the Selma3D 2024 challenges [5], [41]. The BraTS dataset was selected due to its extended use in different publications studying AI applications in organ segmentation, so it serves as a benchmark to sanity check whether the model and SSL techniques work on well-known data. The Selma3D dataset was selected because the focus of the thesis is to analyze how different SSL methods perform on LSFM data, and this is one of the only publicly available datasets that contain quality unannotated and annotated data of this modality.

The BraTS 2021 dataset is a collection of 3D multi-modal MRI images of brain tumors with voxel-level ground-truth labels annotated by physicians. The training set contains 1251 volumes, each of which contains four channels corresponding to different modalities: native (T1), post-contrast T1-weighted (T1ce), T2-weighted (T2) and T2 fluid-attenuated inversion recovery (flair). In addition, for each volume a segmentation mask is included, which contains four labels: 0 for background and non-tumor areas, 1 for the necrotic parts of the tumor, 2 for peritumoral edematous tissue and 4 for enhancing tumor. However, the classes evaluated are the enhancing tumor (label 4), the tumor core (labels 1 and 4) and the whole tumor (labels 1, 2 and 4), so the output segmentation consists of three channels, each with a binary mask segmenting one of the previous classes. Apart from that, a JSON file is provided, which divides the training data into five folds to facilitate reproducibility and cross-validation. The data is saved in NIfTI format (.nii.gz) and has shape 240x240x155; it was preprocessed by aligning the volumes, stripping the skull and resampling the images to a 1x1x1 mm isotropic resolution [4].

The Selma3D 2024 dataset consists of 35 large-scale 3D LSFM images of human and mouse whole brains; it encompasses diverse stained biological structures including blood vessels, c-Fos + cells, cell nuclei and amyloid-beta plaques. It also includes 89 small volumes annotated with binary segmentation masks of the previously mentioned structures [5]. Due to the cellular resolution of LSFM, imaging a whole brain can result in a volume size of several hundreds of gigabytes or even a few terabytes, which makes training and managing several of these volumes impossible for a regular laptop. To overcome this issue, the following approach was taken: each unannotated volume consists of several 2D 16-bit signed TIFF files, so a Python script was created to extract random 250x250x250 subvolumes by cropping and stacking the necessary 2D slides, ensuring

with an Otsu threshold that each subvolume contains significant foreground. This results in manageable information-rich 3D images that can be used for training instead of the original large-scale volumes. Due to time and memory limitations, not all volumes were sampled; the table below contains information about which volumes were sampled for each entity, their original properties and the number of subvolumes taken from each.

Biological entity	Specimen Source	Sample	Resolution (X×Y×Z)	Original Size (X×Y×Z)	Number of Subvolumes
c-Fos+ cells	Mouse brain	Sample 1	1.625 μ m×1.625 μ m×6 μ m	6656×5616×733	500
		Sample 11		6656×5616×721	500
Cell nucleus	Human brain	Sample 1	0.54 μ m×0.54 μ m×5 μ m	19655×11855×834	250
		Sample 2		13478×8133×613	250
		Sample 3		13307×17238×611	250
		Sample 4		16054×16565×768	250
Blood vessels	Mouse brain	Sample 3 C00	1.625 μ m×1.625 μ m×3 μ m	4772×3109×1870	500
		Sample 3 C01		4772×3109×1870	500

TABLE 1 Selma3D data information table [5].

The amyloid-beta plaques were excluded in favor of giving more representation to cells, nuclei and vessels as these entities are usually more present in microscopy data and research. In total, a thousand subvolumes of unannotated data for each entity were obtained, with equal representation for each of the selected samples. For the nucleus, all the provided volumes were sampled as each represented a different region of the brain (hippocampus, motor cortex, sensory cortex and visual cortex). For vessels, two channels were provided for each volume: C00 marks microvessels and C01 marks major vessels. Due to the previously mentioned constraints, only one volume was sampled, with each channel providing diverse representation. For the annotated volumes, they were used as given by the sources; 19 c-Fos+ cell NIfTI patches of size 100x100x100, 24 blood vessel patches of size 500x500x50 with separated files for each channel which share the same segmentation mask, and 12 cell nucleus patches of size 200x200x200.

3.2 Data loading and transformations

A data loading script was created to split the data files into SSL pre-training and fine-tuning sets, with most data going to the former. Each dataset is organized differently, so they have customized data splitting functions. For BraTS, the JSON file is used to do the splitting: the function receives a fold number, and the files contained in that fold are allocated to the fine-tuning set, while the ones contained in the other four are allocated to the pre-training set, using only the raw volumes and not the segmentation files. The fine-tuning set is further divided into training, validation and testing sets following a 70/15/15

partition. There is a parameter that defines what percentage of the fine-tuning training set is actually used for training, to enable comparisons of models using all the data (70%) or fractions of that data for simulating scenarios where labels are scarce.

For Selma, the files are divided into folders called “Annotated” and “Unannotated”, with the first containing the annotations and the associated volumes and the second containing the extracted subvolumes, all organized by subfolders named after the entities and samples they come from. The files are tagged with the entity they belong to, allocating all unannotated files to the pre-training set and the annotated files to the fine-tuning set, ensuring that entities are represented in a balanced way in the 70/15/15 train, validation and test split by using stratification in the `train_test_split` function from `sklearn`. Here, it is also possible to set the fraction of annotated data that is used during supervised training, while maintaining a balanced representation of the entities.

The main dataloader function receives the dataset that must be used to build the dataloaders and the fraction of annotated data to use for training, as well as if it is for SSL pre-training or for fine-tuning among other parameters. Then, it calls the data splitting functions that return which files should be allocated to each split. Only the needed dataloader for the specific training phase is filled; for SSL pre-training only the dataloader with unlabeled data is returned, while for fine-tuning only the train, validation and test dataloaders with annotated data are built. The main data loader function uses Monai’s `Dataset` and `DataLoader` methods to create the four dataloaders.

When using the `Dataset` method, a certain set of transformations is passed depending on which dataloader is being created. There are three types of transformations: for SSL pre-training, for supervised training and for supervised validation/testing. The supervised validation and testing transformations are deterministic and consist of normalizing the image values to the (0, 1) range using percentile-based normalization to avoid outlier influence, ensuring channel-first ordering, and in the case of BraTS doing the conversion of the original multi-class segmentation mask into three binary mask channels corresponding to each class expected from the output (whole tumor, tumor core and enhancing tumor). No stochastic transformations are applied to these splits as they are used to check how the model would perform on regular data without artificial augmentations.

For the supervised training transformations, the previously mentioned transformations are included in combination with data augmentations: first a fixed-size patch (region of interest) is randomly cropped from the original volume, then random flips in each dimension are performed in that patch with a 50% probability, a random intensity scaling in the range (0.9, 1.1) is also performed with an 80% probability, and finally a random per channel intensity shift in the range (-0.1, 0.1) is performed with an 80% probability. For BraTS, there is an additional foreground cropping step before the rest of the data augmentations, as the MRI volumes have background around the imaged brain which does not have valuable information and can therefore be removed. LSFm has sparse elements throughout the whole volume, so it is not possible to apply this step as foreground and background are too intermingled.

For SSL transformations, the following Monai's transformation modules are imported: LoadImaged (loads the image), EnsureChannelFirstd (ensure channel-first ordering), RandSpatialCropd (crops a region of size ROI from the sample), RandFlipd (flips the sample over a given axis), RandScaleIntensityd (scale voxel intensity values), RandShiftIntensityd (shift voxel intensity values), RandGaussianNoised (apply random Gaussian noise), RandRotate90d (rotate the sample 90°), CropForegroundd (crop the foreground region of the sample), ConvertToMultiChannelBasedOnBratsClassesd (converts the BraTS labels to multi-channel), ScaleIntensityRangePercentilesd (normalize the input using percentile-based normalization), ToTensord (converts sample to a tensor).

The data augmentations depend on the selected SSL pre-training method: for the MIM approach, the only transformations needed are the channel-first ordering, percentile-based normalization and random cropping. For BYOL, part of the method is to create two augmented views of the same image, so the transformations for this method are random cropping, random flipping in all axes with 50% probability, 90° rotations in axes that have the same dimension to avoid size mismatch errors, random intensity scaling in the range (0.7, 1.3) and shifting in the range (-0.2, 0.2) with 80% probability, and random Gaussian noise with 50% probability. These transforms are applied to the base image after normalization and ensuring channel-first ordering, getting two different views. Stronger augmentations encourage the model to learn invariant representations in view-based techniques, while reconstruction-based approaches benefit more from minimal augmentations due to their voxel-level learning.

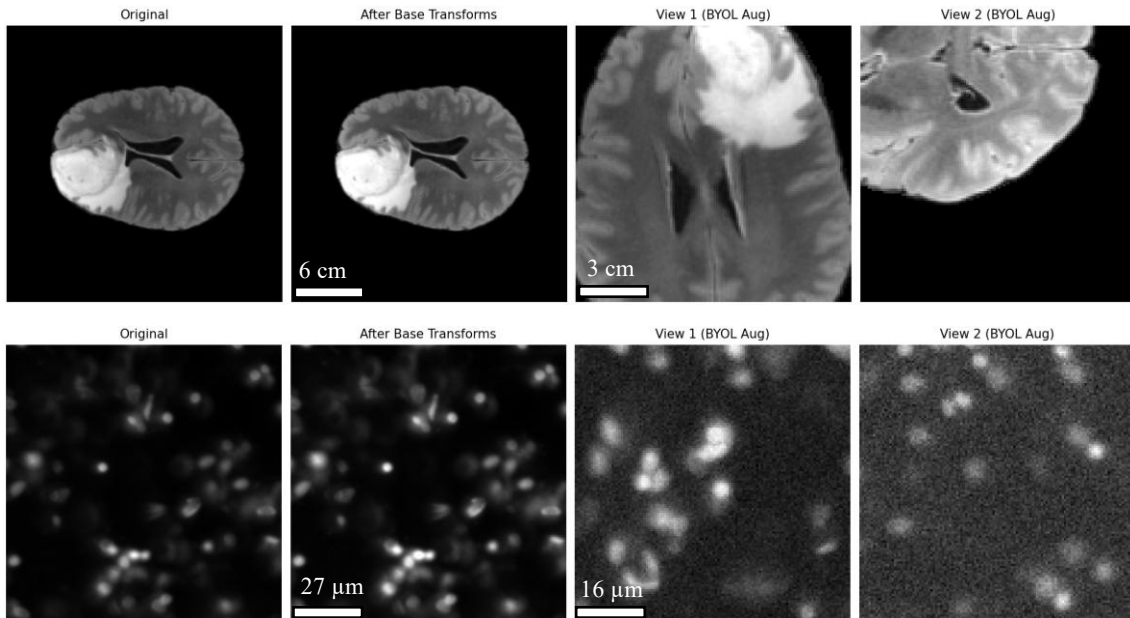


FIGURE 8 Middle slice example of a BraTS flair modality and a Selma cell nucleus volume in their original state, after base transforms (normalization and channel-first ordering), and the two augmented views obtained for BYOL. Scale bars are added for reference to the original size (first two panels) and the cropped size (last two panels), for a ROI of [120, 120, 96].

3.3 Model architecture

The architecture selected for this thesis is SwinUNETR, as it combines the good long-range dependencies modelling and the efficiency to process high-resolution images hierarchically of Swin Transformers, with the strong spatial inductive bias and local feature extraction capabilities of CNNs. This model is specifically designed for 3D images, has achieved SOTA results in organ segmentation in different benchmarks, and has also achieved good performance in microscopy applications, so it was considered a good option for this research [5], [84]. Below, a diagram of the model components is shown.

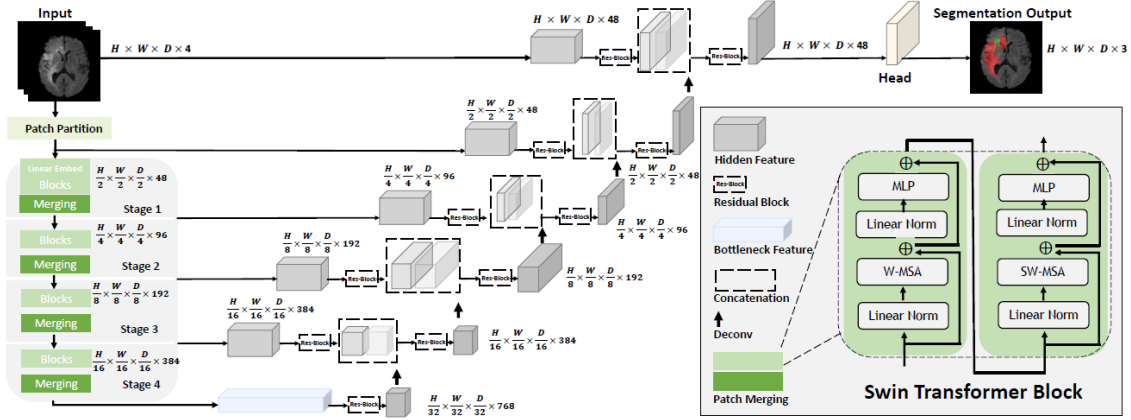


FIGURE 9 SwinUNETR architecture with a BraTS input [84].

First, the 3D input is partitioned into non-overlapping fixed-size patches using a 3D convolution with kernel size and stride equal to the patch size, which also projects the input into a chosen embedding dimension. The output of this patch embedding step is a lower-resolution grid of embedded tokens. These tokens are passed as input to the Swin Transformer stages, which consist of a sequence of two Swin Transformers, the first with standard window-based multi-head self-attention (W-MSA) and the second with shifted window multi-head self-attention (SW-MSA), and a patch merging layer.

To calculate self-attention in an efficient way, the input is divided into non-overlapping fixed-size windows, and the self-attention is calculated independently for the tokens of each window. This reduces significantly computational cost with respect to global attention. In alternating Swin Transformer blocks, the window partitions are shifted by half the window size to enable cross-window interaction (SW-MSA), which enhances context modelling. Apart from the self-attention mechanisms, the Swin Transformer blocks also contain layer normalization (LN), skip connections and a multilayer perceptron (MLP). The equations below define how the outputs of the Swin Transformer blocks are obtained for each layer l using all the previously mentioned components:

$$\hat{z}^l = W - MSA(LN(z^{l-1})) + z^{l-1}$$

$$z^l = MLP(LN(\hat{z}^l)) + \hat{z}^l$$

$$\hat{z}^{l+1} = SW - MSA(LN(z^l)) + z^l$$

$$z^{l+1} = MLP\left(LN(\hat{z}^{l+1})\right) + \hat{z}^{l+1}$$

Where $\hat{z}^l$ and $\hat{z}^{l+1}$ are the outputs of W-MSA and SW-MSA respectively, and z^l and z^{l+1} are the Swin Transformer blocks outputs. The self-attention is calculated with the standard equation:

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

With Q being the query, K the key, V the value, and d the size of the query and key.

The patch merging layers reduce the spatial resolution by half while doubling the channel depth, downsampling the output from the Swin Transformer blocks. This is done to get hierarchical representations of the input in order to enhance feature learning. After the patch merging, a layer normalization is applied and the output from this is saved as a skip connection that will be given to the decoder afterwards. In total, there are four encoding stages, obtaining outputs with spatial resolutions of 1/4th, 1/8th, 1/16th, and 1/32nd the original input. The skip connections are composed of the raw input embedded into the embedding dimension, the partitioned and embedded input (which is half the spatial resolution of the original input), and the outputs of stages one to three, which have the resolution stated above.

Once the encoder output and the skip connections are obtained, they are passed through a residual block which applies two 3x3x3 convolutional layers, instance normalization and LeakyReLU activation. Then, they are concatenated with the upsampled output from the previous layer, passed through another residual block, and upsampled by a factor of 2 themselves. For the deepest level, the bottleneck output is passed through the residual block and upsampled directly, as it is the deepest layer and therefore does not have a deeper output to be concatenated with. This concatenation and upsampling process is done until the original spatial resolution is reached, at which point the decoder output is passed through a segmentation head that projects the feature vector into the final segmentation output.

3.4 Self-supervised pre-training algorithms

The chosen self-supervised techniques are MIM, a generative method, and BYOL, a non-contrastive self-distillation method. These techniques were chosen because they have both been used to pre-train models used for medical image segmentation, improving their performance and achieving SOTA results in benchmarks like the BTCV. They have also been used in the Selma3D challenge, achieving relatively good results [5], [51], [52], [54], [85]. Another reason for their selection is that this thesis aims to analyze the impact of SSL pre-training on LSFm data, therefore having two significantly different approaches is valuable.

3.4.1 MIM approach

The MIM approach starts by using SwinUNETR's patch embedding function to project the 3D input into the embedding dimension, where a mask is applied hiding the defined

ratio of patches. A patch-level masking approach is used because the embedding dimension is more expressive than the original voxel-level space. Afterwards, the masked patches are replaced by learnable mask tokens, so the model can learn how to reconstruct them. The visible patches along with the learnable tokens go through the SwinUNETR encoder, consisting of the Swin Transformer stages and patch merging layers.

The skips connections and the encoder output are obtained, processed through a ResBlock as explained above and fed into the SwinUNETR decoder, which outputs a reconstruction of the original spatial resolution. Then, this representation is passed through a reconstruction head consisting of a convolutional layer that projects the reconstruction to the original channel dimension and applies a sigmoid to obtain voxel values in the original $[0, 1]$ range. Both the reconstruction and the mask upsampled to voxel level are returned by the MIM pipeline, as the mask is needed to calculate the reconstruction loss.

Some papers use a simple lightweight decoder without skip connections for MIM pre-training, which is then discarded and replaced by the heavy segmentation decoder which is trained from scratch. However, this approach did not yield good reconstructions in the experiments nor seemed to impact downstream performance, so in the end the original full SwinUNETR structure was used for the pre-training, keeping the pre-trained weights of the encoder and decoder stages for the downstream task and discarding only the reconstruction head and the learnable mask tokens.

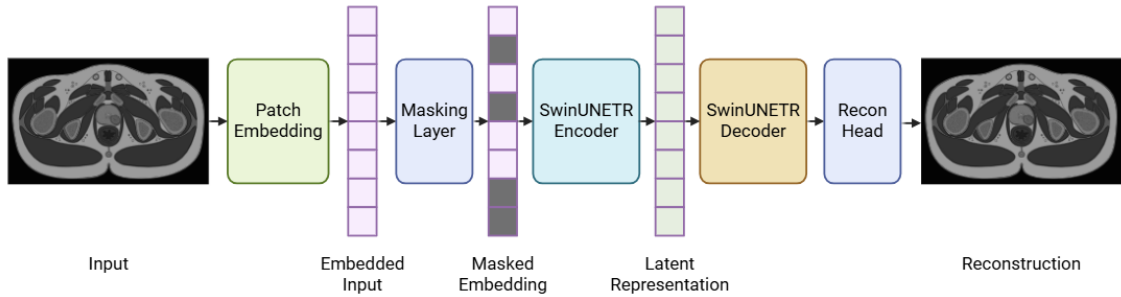


FIGURE 10 MIM schematic diagram showing the encoding, masking and reconstruction pipeline.

3.4.2 BYOL approach

The BYOL approach [23] requires two neural networks to learn, the online and the target networks. The online network consists of an encoder f_θ , a projector g_θ and a predictor q_θ ; the target network has the same architecture as the online network but with different weights, and without a predictor stage. The encoder used is the previously described SwinUNETR encoder, which is composed of the patch embedding layer, Swin transformer stages and patch merging layers. After obtaining the encoder output, the 3D feature map is flattened into a single 1D vector per sample, normalized and pooled.

The projector stage consists of an MLP composed of a linear layer, a layer normalization, a ReLU activation and another linear layer. The first linear layer projects the encoder output with embedding dimension 1536 to a hidden space of size 1024. Layer normalization is applied instead of batch normalization for stability, as the batches were restricted to small sizes due to computational constraints. The last linear layer projects

the representation from the hidden space to a projection of size 256, to obtain a more compact representation.

The predictor stage also consists of an MLP with a linear layer, a layer normalization, a ReLU activation and another linear layer. The first linear layer takes the projection output and embeds it into a hidden space of size 1024, passes it through the normalization and activation layers and then projects it back to the original size of 256. After the encoder, projection, and prediction architectures are initialized, the target network copies the projection stage and the encoder architecture from the online network.

BYOL takes two augmented views from a sample: both are encoded by the target and online encoders separately. The resulting representations are then passed to the online and target projection stages, obtaining a lower-dimensional projection of the views. Then, the online predictor takes the online projection of each view and tries to output a representation that matches the target projection of the other view. After each training step, the online network is updated through backpropagation, but the target network is updated through an exponential moving average of the online network's weights. This is done to prevent a collapsed constant representation by keeping the target network as a stable, fixed target producer with respect to the online network. Below is a diagram of the process explained above for clarity.

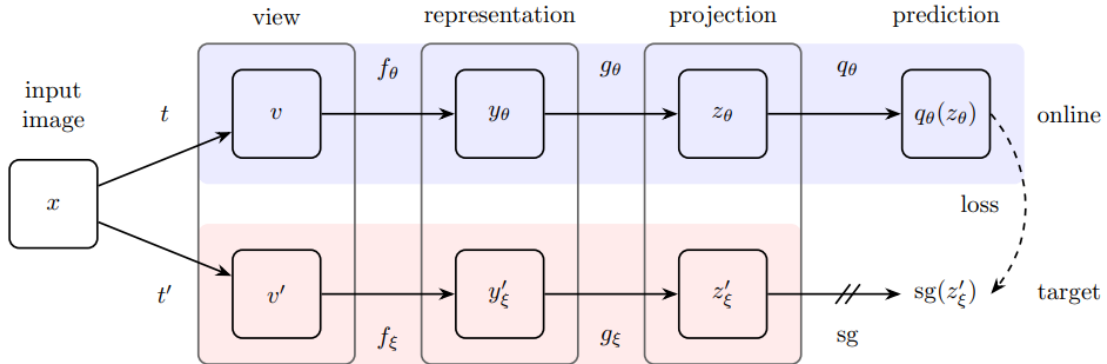


FIGURE 11 BYOL's architecture. BYOL minimizes a similarity loss between $q_\theta(z_\theta)$ and $sg(z'_\xi)$, where θ are the trained weights, ξ are an exponential moving average of θ and sg means stop-gradient. At the end of training, everything but f_θ is discarded, and y_θ is used as the image representation [23].

After pre-training with BYOL, all stages are discarded except for the online encoder, whose weights are preserved for transferring on the downstream task. As BYOL does not use a decoder, only the SwinUNETR encoder benefits from this approach, unlike with MIM which transfers both encoder and decoder weights.

3.5 Supervised and self-supervised training

Custom scripts were made for the self-supervised pre-training and the supervised training. Both contain a main script that handles the parameter definition, data loading, model initialization, checkpoint organization and trainer definition. First, the hyperparameters are defined depending on the arguments passed to the script; below is a table with the used hyperparameters, their definition and the default value which will be used if not

given one. A hyperparameter with the default value “None” means that hyperparameter is not used in that type of training.

Argument	Flag	Default value		Definition
		SSL pre-training	Supervised training	
Epochs	--epochs	100	100	Number of epochs for the training
Learning rate	--lr	5e-4	1e-4	Initial learning rate for the training
Batch size	--batch	2	1	Number of samples used in a training iteration
Embedding dimension	--embed_dim	96	96	Size of the feature representation vector
Fold	--fold	1	1	BraTS data fold used for fine-tuning
ROI	--roi	[96, 96, 96]	[96, 96, 96]	Size of the cropped area taken from a data sample
Validation frequency	--val_every	10	None	Defines after how many epochs a validation is done
Training data fraction	--train_fraction	None	1.0	Fraction of the fine-tuning training data used
Accumulation steps	--accum_steps	8	None	After how many batches are the model weights updated
Mask ratio	--mask_ratio	0.75	None	Fraction of the sample that is masked for MIM pre-training

TABLE 2 Training hyperparameters.

There are also parameters that are fixed inside the script for optimal performance: the patch size is set to (2, 2, 2), the window size is set to (8, 8, 8) and the number of heads for each Transformer stage is set to 3, 6, 12, and 24 respectively. Apart from these hyperparameters, each script has additional arguments used for checkpointing or for selecting different functionalities of the training. Both SSL and supervised training scripts require an experiment number to be given for managing the checkpoint folder (--experiment) and can optionally take an argument to resume training from a previous checkpoint file called “last.ckpt” (--resume_dir), in case the training was interrupted.

The scripts also take an argument defining which dataset should be used for training (--data), “brats” or “selma”. The SSL script can additionally take an argument to select which pre-training method should be used, “mim” or “byol” (--ssl_mode), and the

supervised script can take an argument for loading a pre-trained checkpoint (`--pretrain_ckpt`); if no pre-trained checkpoint is given, the supervised training will be done from scratch. This enables the comparison between a purely supervised approach and an SSL fine-tuning approach.

After defining the arguments, the data is loaded by calling the data loading function for the selected dataset, and the number of input and output channels is set. BraTS has four input channels (the MRI modalities) and Selma has one input channel (raw volume). For supervised training, the output channels must match the expected segmentation classes: BraTS uses three output channels (the three tumor regions), and Selma uses one output channel (one class for foreground). For self-supervised pre-training, the output channel argument is not used, as MIM aims to reconstruct the input and BYOL deals with encoded representations, they do not perform segmentation.

Then, the model is initialized with the given parameters and wrapped in a Lightning module, which organizes all the code needed for deep learning training into a single, self-contained structure. This includes model architecture and weights, training/validation/test loops, optimizers, loggers and more, to make the code scalable, reproducible and portable. The checkpoints are handled by Lightning's CSVLogger and ModelCheckpoint classes; the CSVLogger logs the metrics of the training and saves them to a csv file, and the ModelCheckpoint saves .ckpt files following a set of given rules.

The logs and checkpoints are saved in a folder defined by the experiment number and the data: `Experiment_X/data_Y`, and each run creates a new version folder inside (`version_Z`). If the `--resume_dir` argument is passed with a valid path to a folder which contains a `last.ckpt` file, the logger and checkpoints will be created in a folder called `version_Z_cont`, to indicate from which version it is taking the checkpoint. Two types of checkpoints are saved: the `last.ckpt` checkpoint is overwritten every five epochs, to keep a recent version of the training progress, and the `best-model` checkpoint is overwritten when a model obtains a better training metric (training loss for SSL pre-training and validation score for supervised training) than the last model saved, to keep track of the best performing model.

The training process is handled by Lightning's Trainer, which takes arguments regarding the number of epochs for training, the logger and checkpoints to be managed, and settings for computation efficiency like which device to use and which numerical precision. The training begins when the Trainer is called at the end of the main script and given the data and model to use. An AdamW optimizer with a weight decay of $1e-5$, and a cosine annealing learning rate scheduler are used. The main scripts of self-supervised and supervised training go through the same previously explained steps, but the training structure and strategy defined inside the Lightning module that wraps the model is different.

3.5.1 Supervised training

The metrics used to track the performance of supervised training are the Dice score and the Jaccard index, also known as the intersection over union. These are typically used in medical segmentation tasks [86], [87], as medical datasets usually have small foreground

regions and are dominated by background, creating class imbalances that affect training. Dice and Jaccard focus on measuring the overlap between the predictions and the ground truth, ignoring true negatives and overcoming background bias.

For the BraTS dataset, a pure Dice-based loss is used, and for the Selma dataset a combination of Dice and cross-entropy loss is used, due to LSFV volumes having sparse small elements in a large background and requiring a more robust and stable loss. Below are the equations to calculate the Dice and Jaccard scores [86]:

$$Dice = \frac{2 * TP}{FP + 2 * TP + FN} \quad Jaccard = \frac{TP}{FN + TP + FP}$$

In the context of segmentation, true positive (TP) refers to the number of voxels that are correctly classified as their corresponding class, false positive (FP) refers to the number of voxels classified as belonging to a certain class when they do not, and false negative (FN) refers to the number of voxels that are classified as not belonging to a certain class when they do. The Jaccard index penalizes false positives and negatives more heavily than Dice, serving as a stricter benchmark for segmentation quality.

The supervised training pipeline is the following: a batch of samples is loaded, and a forward pass is performed with the model to obtain raw logits. These logits are passed to the corresponding loss function to obtain the training loss, which is logged for each epoch and used for backpropagation. Every 10 epochs, a validation epoch is performed; Monai's sliding window inference is used to split the full volume into smaller windows where predictions are performed independently and then stitched together for memory-safe computation. Once the logits are obtained, they are used to calculate the validation loss. Then, they are converted into probabilities using a sigmoid function and turned into a binary prediction using a threshold. These predictions are then passed to the metric functions to obtain the Dice score and the Jaccard index per sample.

After all the validation samples have been processed, the metrics and losses are aggregated across all samples to obtain the average Dice score, Jaccard index and validation loss, which are all logged. The Dice score is calculated for each class, so in the case of BraTS, there are three Dice scores, one for each tumor region, and a global Dice score which encompasses all classes. The Jaccard index is only calculated globally for simplicity. At the end of the training, the designated folder contains the best and last model checkpoints, a csv file with the metrics and losses, and a yaml file with the used hyperparameters.

3.5.2 Self-supervised training

The self-supervised pre-training pipeline depends on the SSL method chosen. When the training starts, the model is initialized to either the MIM class or the BYOL class depending on the argument passed. If the selected SSL mode is MIM, the trainer loads a batch and passes the samples as input to the model, which returns the upsampled mask used and the reconstruction obtained. The loss is obtained by applying the mask to both the reconstruction and the original sample and calculating the L1 loss and SSIM loss [88] between the masked patches only. The losses are defined as:

$$L1 = |x_n - y_n| \quad SSIM = \frac{(2\mu_A\mu_B + C_1)(2\sigma_{AB} + C_2)}{(\mu_A^2 + \mu_B^2 + C_1)(\sigma_A^2 + \sigma_B^2 + C_2)}$$

Where x_n is the input, y_n the ground truth, A and B are two input volumes, μ_A and μ_B are the means, σ_A^2 and σ_B^2 are the variances, σ_{AB} is the covariance, and C_1 and C_2 are constants based on stability terms: $C_1 = K_1^2$ and $C_2 = K_2^2$, with $K_1 = 0.01$ and $K_2 = 0.03$ when the input image is in the range $[0, 1]$ as in this case. L1 is the mean absolute error, which is usually used in medical segmentation in place of the mean squared error due to being more robust to outliers and giving sharper edges. This loss focuses on voxels having the correct intensity values. SSIM is the structural similarity index measure, which focuses on how similar luminance, contrast and structure in the volumes are, to make the model learn global structure.

The total loss is then calculated by adding both losses, with the SSIM loss being weighted so that it does not dominate the training due to its larger and less stable value. Due to how SSIM is calculated, it is possible to obtain NaN or Inf values if the masked regions have very low variance or the sampled patch has no foreground. To avoid the crash of the training, the loss falls back to using only L1 in case the SSIM loss fails. The total loss, L1 loss and SSIM loss are logged per epoch. Once every 10 epochs, the last batch of the epoch is passed to a visualization code that plots the middle slice of the original volume, masked volume and reconstruction and saves the figure to the checkpoint folder. This is done to keep a qualitative track of how the model reconstructs over time.

For the BYOL mode, the trainer loads a batch which contains two augmented views of the original sample and passes those views to the model. These views are used to obtain the target and predicted representations of both given by the target and online networks, which are then used to calculate the loss. This loss is symmetrical, as the predicted representation of view 1 is compared to the target representation of view 2 and vice versa, in order to achieve augmentation-invariant features. First, the predictions and targets for each view are l_2 -normalized, and then the cosine similarity between the online prediction of view 1 and the target representation of view 2 is calculated, as well as the cosine similarity between the online prediction of view 2 and the target representation of view 1. Then, these losses are added together to obtain the final loss, which is averaged across batches and logged per epoch. Below is the BYOL loss [23]:

$$loss = 2 - 2 \frac{\langle q_\theta(z_\theta), z'_\xi \rangle}{\|q_\theta(z_\theta)\|_2 \cdot \|z'_\xi\|_2}$$

Where $q_\theta(z_\theta)$ is the online prediction of z'_ξ , which is the target representation. After each training step ends, the target network is updated following this equation:

$$\xi \leftarrow \tau \xi + (1 - \tau) \theta$$

With ξ being the target network parameters, θ the online network parameters, and τ the EMA decay. The τ is adapted during training, with a smaller decay at the beginning to speed up training which increases by following a cosine EMA schedule, ramping up the

moving average decay from the initial value of 0.9995 to 1.0 over training. In a cosine EMA schedule, the τ is updated like this [89]:

$$\tau \leftarrow 1 - (1 - \tau_0) \cdot \frac{(\cos(\frac{\pi k}{K}) + 1)}{2}$$

Where τ_0 is the initial decay rate, k is the current training step and K is the total training steps.

3.6 Testing

The final models obtained through SSL pre-training and fine-tuning or through regular supervised training can be tested on previously unseen samples using a custom testing script. This code takes a model checkpoint, the dataset it was trained on and the type of test to perform: a qualitative visualization test (--test_type “visualize”) or a quantitative metrics test (--test_type “metrics”). The visualization test loads a given number of samples from the test dataloader, which is not used during training nor validation, performs a forward pass of the loaded model on them, and plots the middle slices of the original volume, the ground truth and the model segmentation prediction together.

The metrics test takes all the samples of the test dataloader and performs a forward pass of the loaded model on them, comparing the ground truth and the segmentation prediction using the same metrics as the training: the Dice score and the Jaccard index. This script also obtains further information of the performance compared to the training and validation; it calculates the mean and standard deviation of the Dice score and Jaccard index, and the mean test loss for the test data in general. It also calculates metrics per-class in the case of BraTS and per-entity in the case of Selma. Therefore, it measures what the average Dice score is for each tumor class, and what the average Dice score and Jaccard index is for each Selma entity (cells, nuclei and vessels). After all test samples are processed, the metrics obtained are saved to a csv file for tracking.

As the full 3D volumes are too big compared to the ROI patches the model was trained in, the approach to test the model’s performance is to slide a window of size ROI through the whole volume, testing the model on each window independently and then stitching the predictions together. To avoid artifacts and low performance on the window boundaries, the windows are shifted so that they have a 50% overlap with each other; this way the voxels on the boundaries have multiple predictions that are averaged together to get a quality prediction.

This process is done by setting the loaded model to eval mode and using Monai’s sliding window inference, which deploys the model in the manner described above. After obtaining the predictions, the logits are converted to the [0, 1] range using sigmoid, and the binary segmentation mask for each output channel is obtained by thresholding, so that voxels with over 50% probability of belonging to the class are converted to 1s and otherwise converted to 0s.

4 RESULTS

This section explains the experiments performed during the thesis, which aim to analyze the impact of self-supervised pre-training in the downstream segmentation task. Even though the focus of this thesis is on LSFM (Selma data), the experiments were also performed on BraTS due to it being a popular benchmark for both supervised and self-supervised approaches. The BraTS experiment results can then be compared to existing studies to check if the methodologies work as expected or not, and to check which factors could potentially be the cause of unexpected results. This could not be done if only Selma was used due to the lack of research in the SSL for LSFM area. The hyperparameters used during the experiments are the default ones specified in Table 2 and the paragraph below, except for the experiments which use a reduced dataset, in which case the `--train_fraction` argument is specified: 10% of data is obtained with 0.1 fraction and 50% of data is obtained with 0.5 fraction.

4.1 Supervised training results

First, a standard supervised training was done, to both check that the model architecture works well and to obtain a baseline performance in segmentation for later comparison with the self-supervised approach. For this, the SwinUNETR model was initialized with random weights and trained and validated on the respective full annotated train and validation splits, which contain 175 train samples and 37 validation samples for BraTS, and 55 train samples and 12 validation samples for Selma. Below, the metrics and losses obtained from the supervised training of each dataset are shown.

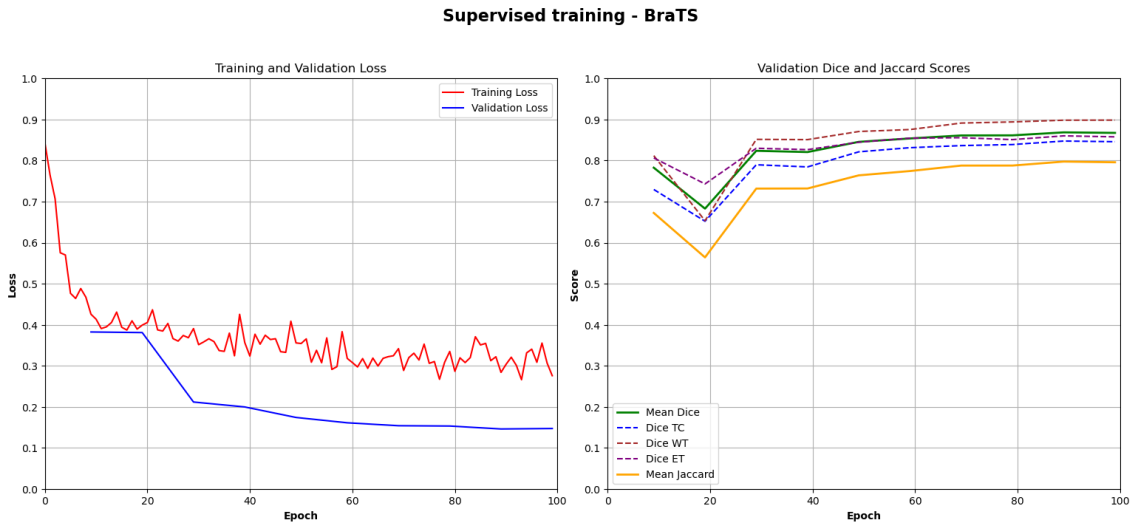


FIGURE 12 Supervised training from scratch for BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using the full annotated data splits. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset.

For BraTS, the training and validation losses have a steeper decrease at the initial epochs, to then stabilize at around 0.3 for the training loss and 0.15 for the validation loss. The Dice scores and Jaccard index are lower in the initial validation checks, increasing in

value at the epoch 30 check and achieving a stable small increase in the following validation epochs, reaching values of around 0.85-0.9 for the Dice scores and 0.8 for the Jaccard index.

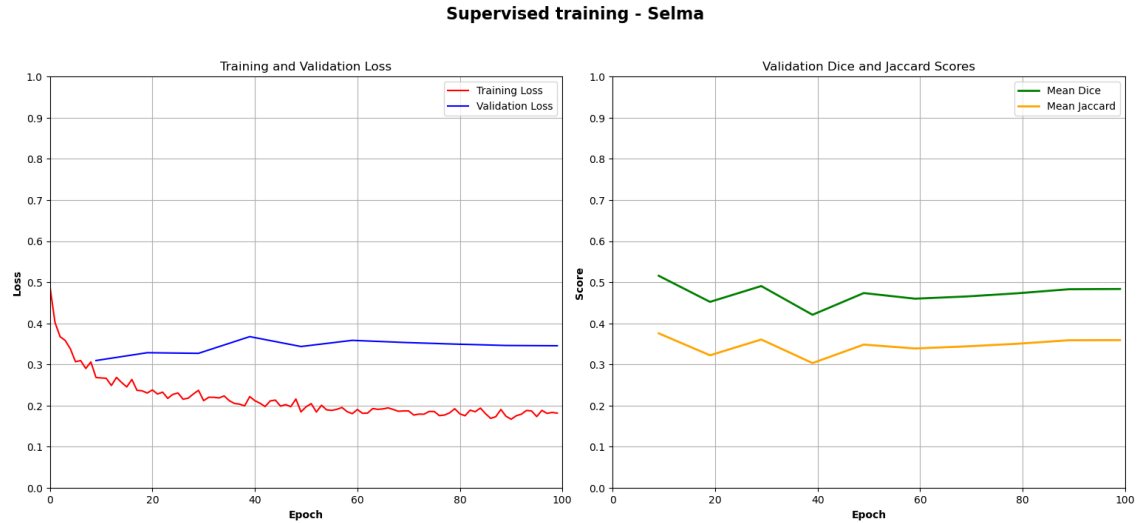


FIGURE 13 Supervised training from scratch for Selma data. The graph on the left shows the training and validation losses for the Selma dataset using the full annotated data splits. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset.

For Selma, the training loss also follows a steeper decrease at the initial epochs, to then stabilize at around 0.2 at later epochs. For the validation loss, no decrease is observed; the loss value is maintained at approximately 0.35 throughout the whole training, with small fluctuations up and down. The Dice score and Jaccard index also have no apparent increase across the validation checks, fluctuating between 0.4-0.5 for Dice and 0.3-0.4 for the Jaccard index.

Self-supervised pre-training is especially useful in low-data regimes, so the performance for the supervised training was also measured when using smaller annotated data fractions for comparison. For BraTS, only 10% of the train dataloader data was used, which consists of 17 training samples. For Selma, 50% of the train dataloader data was used, which consists of 27 training samples. These percentages were selected with the aim of obtaining enough samples to have some diversity but not enough for the model to learn as well. The validation dataloader is not reduced, so the metrics can be compared across experiments. Below are the metrics and losses obtained from these low-data experiments.

Supervised training 10% data - BraTS

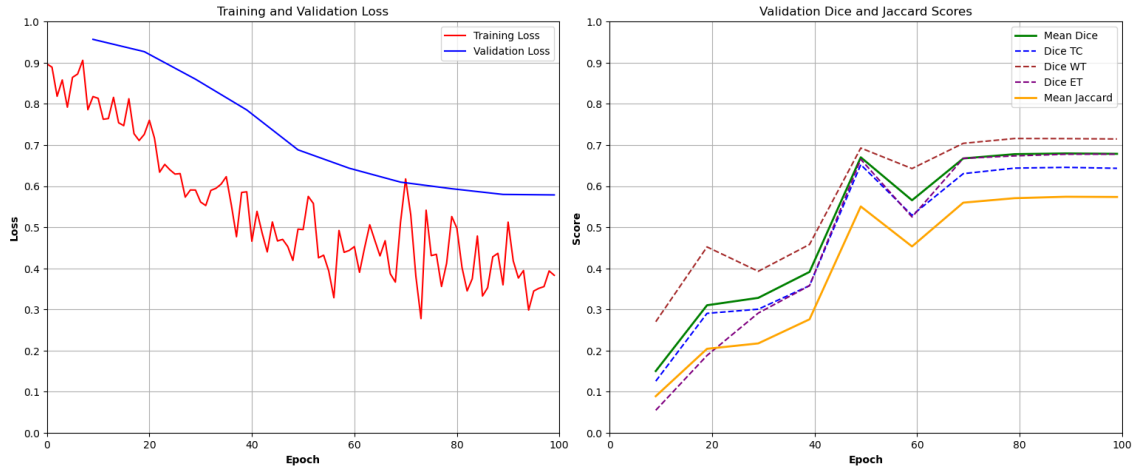


FIGURE 14 Supervised training from scratch for a reduced BraTS dataset. The graph on the left shows the training and validation losses using 10% of the annotated training data split. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints using the full validation split.

In the reduced data supervised training for BraTS, the training and validation losses have a smooth decrease along the epochs, starting at values higher than 0.9 and gradually decreasing until reaching a training loss of around 0.4 and a validation loss of around 0.6. The Dice scores and Jaccard index are low in the initial validation checks, increasing in value gradually until the epoch 70 check, where they reach a stable value of around 0.65-0.7 for Dice and 0.57 for Jaccard.

Supervised training 50% data - Selma

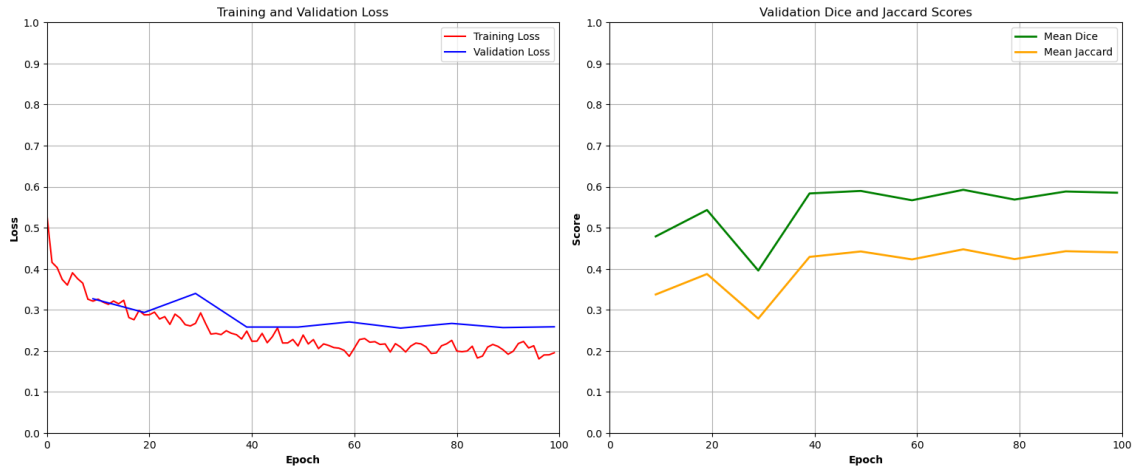


FIGURE 15 Supervised training from scratch for a reduced Selma dataset. The graph on the left shows the training and validation losses using 50% of the annotated training data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints using the full validation split.

In the reduced data supervised training for Selma, the training loss has a smooth decrease along the epochs, from an initial value of 0.5 to a final value of 0.2. The validation loss has a small decrease from an initial value of 0.32 to a final value of 0.26; however, it is

not smooth and gradual, but with fluctuations. The Dice score and Jaccard index suffer fluctuations at the initial validation checks, reaching a higher, stable value of 0.6 for Dice and 0.43 for Jaccard at around epoch 40.

4.2 MIM pre-training results

Two models were pre-trained using the MIM approach, one for the BraTS dataset and another for the Selma dataset. For this, a SwinUNETR model wrapped by the MIM class was initialized with random weights and trained on the full pre-training dataloader, which contains 1001 unannotated train samples for BraTS, and 3000 unannotated train samples for Selma. Below, the total reconstruction losses obtained from the MIM pre-training of each dataset are shown.

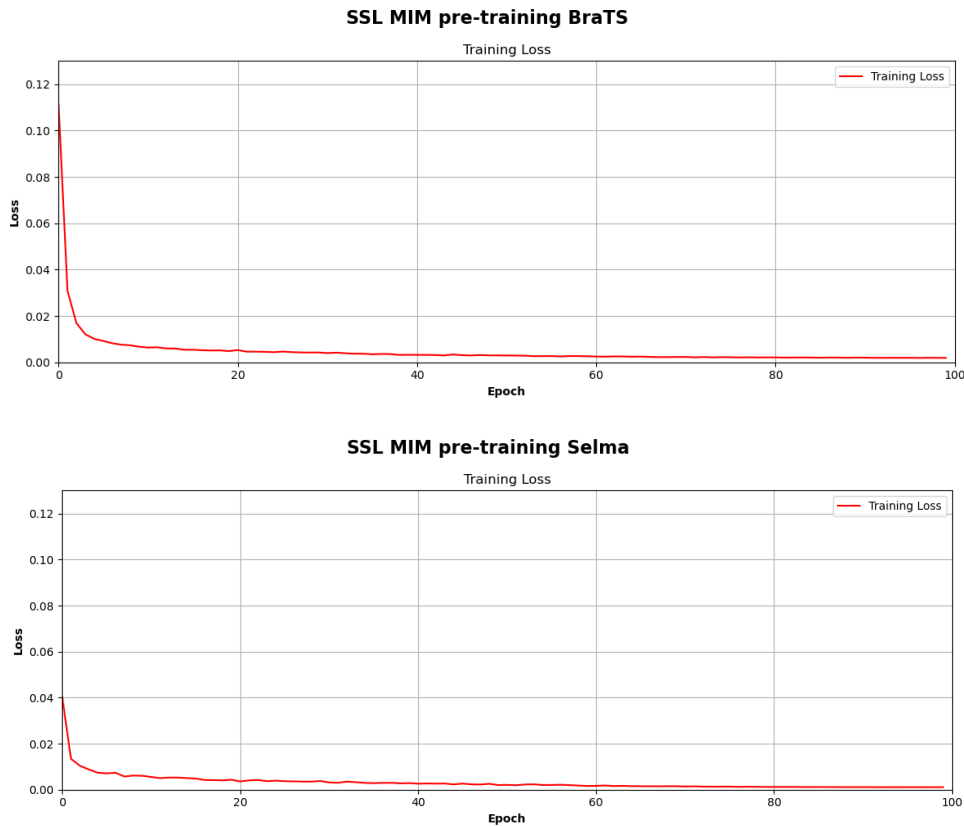


FIGURE 16 MIM pre-training losses across 100 epochs for BraTS (graph above) and Selma (graph below).

For both BraTS and Selma MIM pre-training, the reconstruction loss rapidly decreases in the initial epochs, and then starts decreasing more gradually but continuously, going from an initial value of 0.11 for BraTS and 0.04 for Selma to a final value lower than 0.001 for both.

Apart from the pre-training loss, in MIM approaches it is also valuable to check the reconstructions obtained from the model, to see if it is learning spatial features properly and managing to reconstruct the input, or if it is defaulting to reconstructing noise or average values. Below, a comparison of the original sample, the sample with the upsampled mask applied and the reconstruction obtained is shown for each dataset.

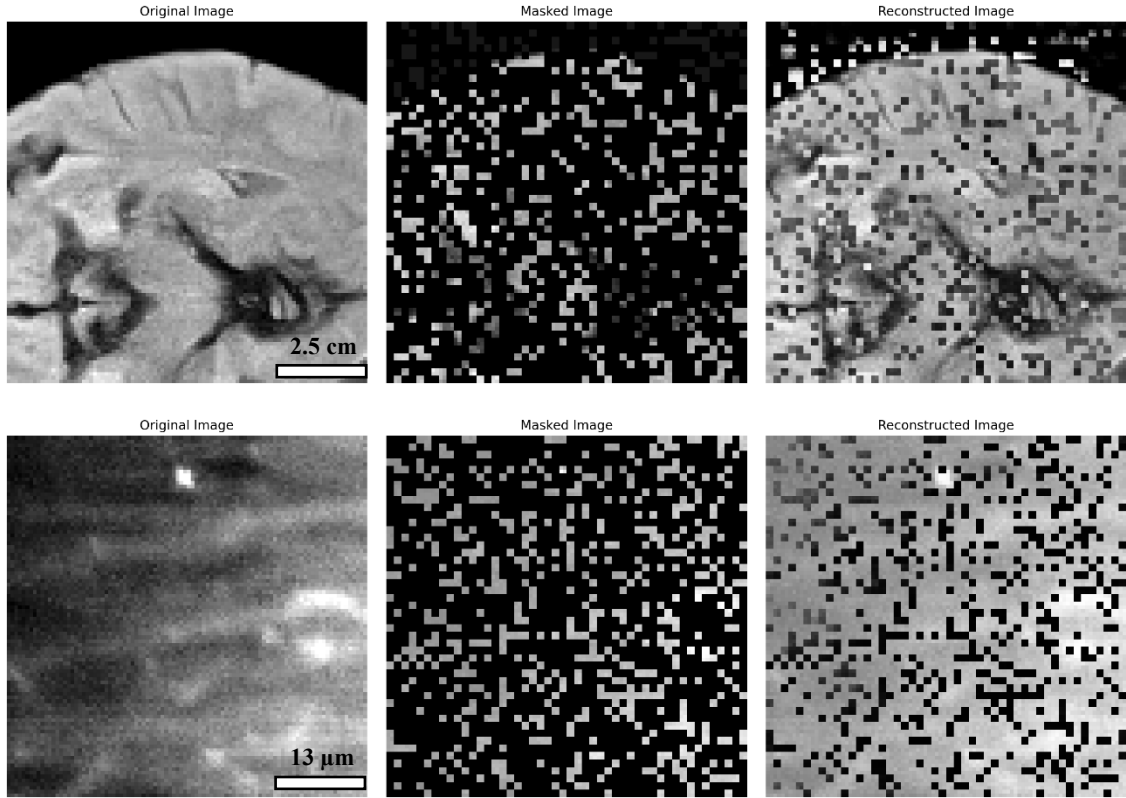


FIGURE 17 This figure shows the middle slices of a processed cropped sample given as input to the model, the same sample with the mask applied at voxel-level, and the reconstruction obtained by the model. The panels above show the visualizations for a BraTS sample, and the panels below show the visualizations for a Selma sample. The mask applied in both cases covers 75% of the sample, and scale bars are added for clarity.

4.3 BYOL pre-training results

As with the MIM approach, two models were pre-trained using the BYOL approach, one for the BraTS dataset and another for the Selma dataset. For this, a SwinUNETR model wrapped by the BYOL class was initialized with random weights and trained on the full pre-training dataloader, which contains 1001 unannotated train samples for BraTS, and 3000 unannotated train samples for Selma, as explained above. Below, the total BYOL losses obtained from the BYOL pre-training of each dataset are shown. Due to time constraints, the BYOL pre-training for the Selma dataset could only be done until around epoch 50, at which point the loss had a value of $3e-6$, which was deemed acceptable.

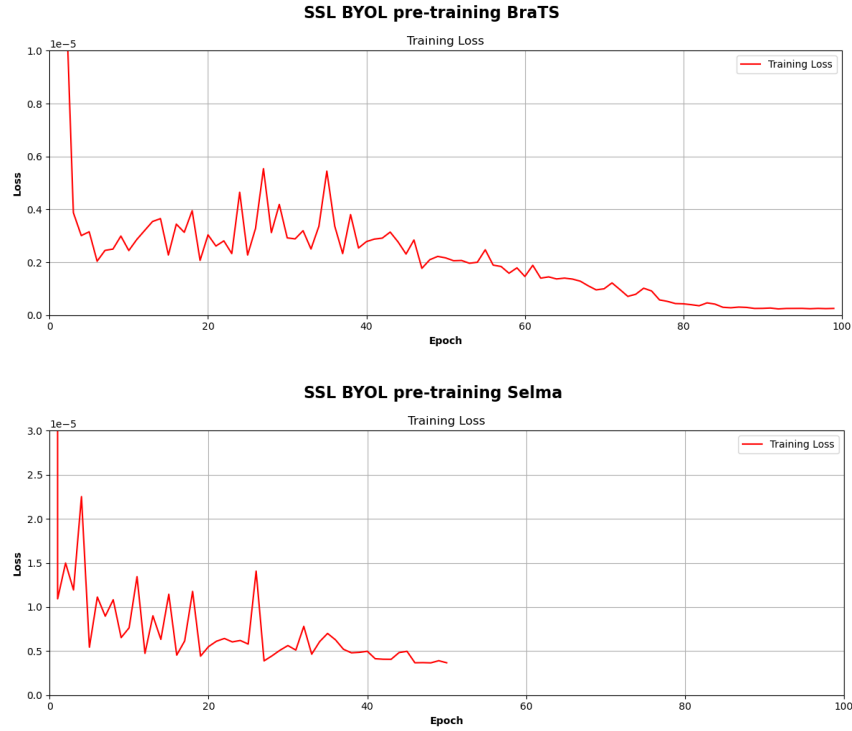


FIGURE 18 BYOL pre-training losses across 100 epochs for BraTS (graph above) and 50 epochs for Selma (graph below).

The BYOL pre-training loss follows a similar trend for both BraTS and Selma: initially the loss decreases dramatically to below $1e-4$ value, and then fluctuates up and down across the epochs but with a slight downwards trend, decreasing to around $2e-7$ for BraTS and $3e-6$ for Selma.

4.4 MIM fine-tuning results

The encoder, ResBlock and decoder weights pre-trained with MIM were transferred to the downstream task and fine-tuned using a supervised approach to adapt the model to learn segmentation. To compare the fine-tuning performance to the supervised approach in low-data regimes, the MIM pre-trained models were fine-tuned both on the full annotated data train split and on the reduced annotated data train split. For the BraTS pre-trained model, only 10% of the train dataloader data was used on the reduced training, which consists of 17 training samples. For Selma, 50% of the train dataloader data was used on the reduced training, which consists of 27 training samples. Again, the validation dataloader is not reduced, so the metrics can be compared across experiments. Below are the metrics and losses obtained from both the full data fine-tuning and the reduced data fine-tuning of the MIM pre-trained models for each dataset.

Fine-tuning from MIM - BraTS

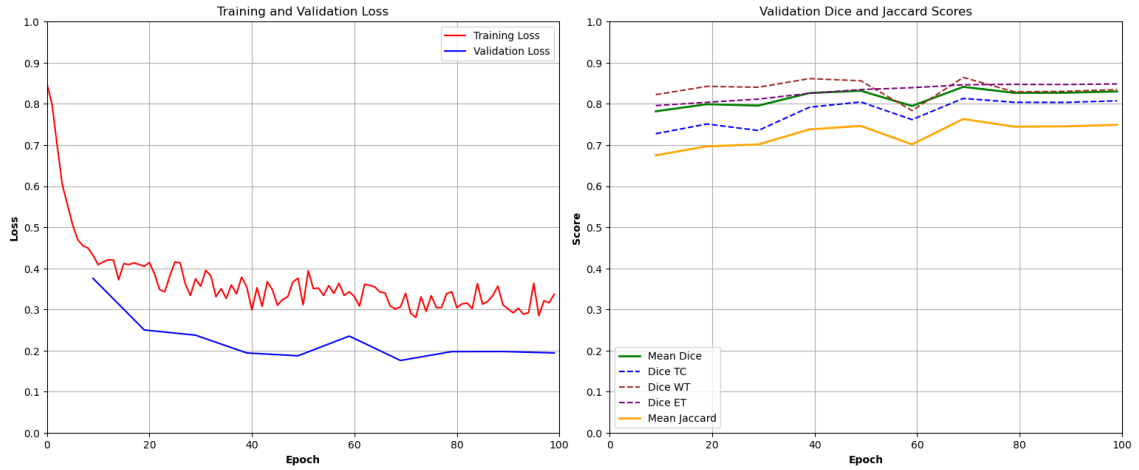


FIGURE 19 Fine-tuning of the MIM pre-trained model for full BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using the full training data. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset.

For BraTS fine-tuning from MIM pre-training, the training and validation losses have a steeper decrease at the initial epochs, to then stabilize at around 0.3 for the training loss and 0.2 for the validation loss. The Dice scores and Jaccard index increase slightly across the epochs, reaching values of around 0.8-0.85 for the Dice scores and 0.75 for the Jaccard index.

Fine-tuning from MIM 10% data - BraTS

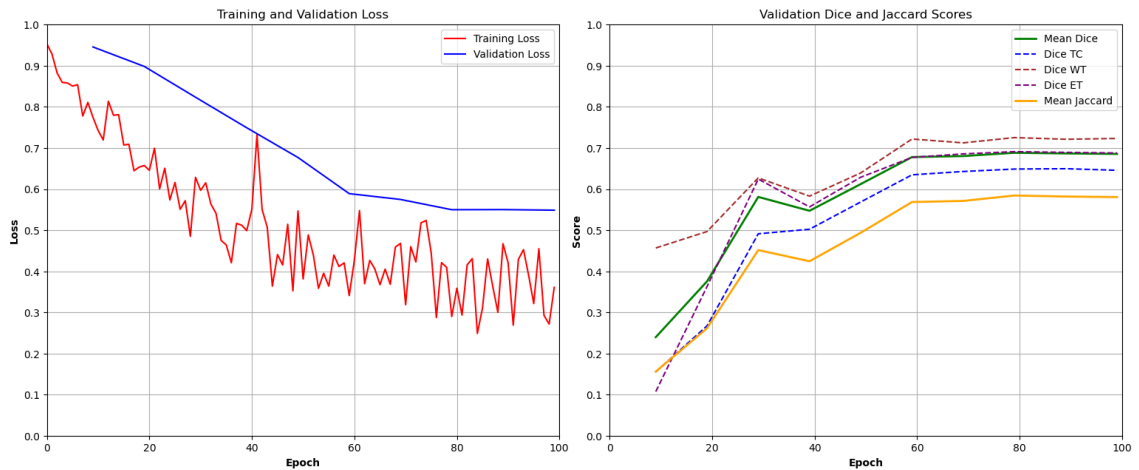


FIGURE 20 Fine-tuning of the MIM pre-trained model for reduced BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using 10% of the training split. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset.

When reducing the training data for BraTS MIM pre-trained fine-tuning to 10% of the original amount, the training and validation losses obtained start at an initial value of 0.95 each, gradually decreasing until reaching a value of 0.35 for the training loss and 0.55 for the validation loss. The Dice scores and Jaccard index start at low values, increasing

gradually until epoch 60, where they achieve a relatively stable value of around 0.65-0.72 for the Dice scores and 0.59 for the Jaccard index.

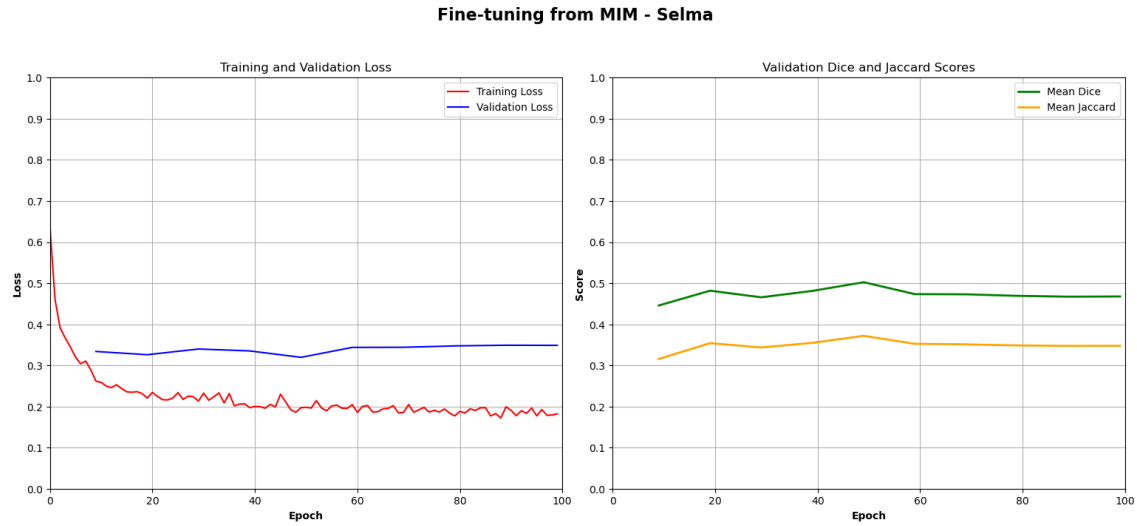


FIGURE 21 Fine-tuning of the MIM pre-trained model for full Selma data. The graph on the left shows the training and validation losses for the Selma dataset using the full annotated data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset.

For Selma fine-tuning from MIM pre-training, the training loss also follows a steeper decrease at the initial epochs, to then stabilize at around 0.2 at later epochs. For the validation loss, no decrease is observed; the loss value is maintained at approximately 0.35 throughout the whole training, with small fluctuations up and down. The Dice score and Jaccard index also have no apparent increase across the validation checks, fluctuating between 0.45-0.5 for Dice and 0.3-0.35 for the Jaccard index.

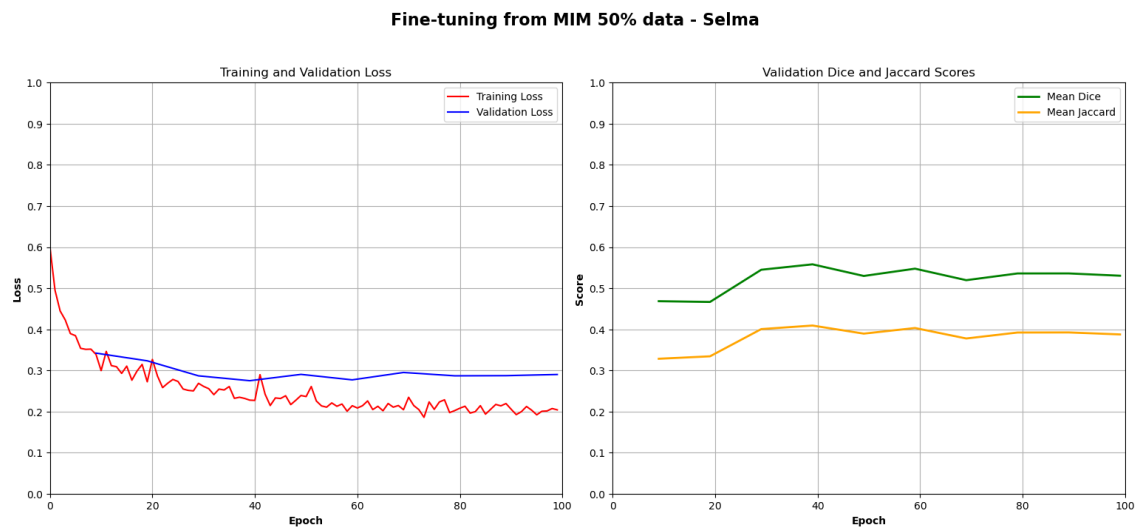


FIGURE 22 Fine-tuning of the MIM pre-trained model for reduced Selma data. The graph on the left shows the training and validation losses for the Selma dataset using 50% of the training data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset.

When reducing the training data for Selma MIM pre-trained fine-tuning to 50% of the original amount, the training and validation losses obtained start at an initial value of 0.6 and 0.35 respectively, with the training loss gradually decreasing until reaching a value of 0.2 and the validation loss having a slight decrease to a value of 0.3. The Dice score and Jaccard index gradually increase across the validation checks, obtaining a relatively stable value of 0.53 and 0.4 respectively after epoch 60.

4.5 BYOL fine-tuning results

The encoder weights pre-trained with BYOL were transferred to the downstream task and fine-tuned using a supervised approach to adapt the model to learn segmentation. Due to the decoder weights not being pre-trained with BYOL, they had to be trained from scratch during fine-tuning. As with the MIM fine-tuning approach, the BYOL pre-trained models were fine-tuned both on the full annotated data train split and on the reduced annotated data train split. Similarly, for the BraTS pre-trained model, only 10% of the train dataloader data was used on the reduced training, and for Selma, 50% of the train dataloader data was used. Again, the validation dataloader is not reduced, so the metrics can be compared across experiments. Below are the metrics and losses obtained from both the full data fine-tuning and the reduced data fine-tuning of the BYOL pre-trained models for each dataset.

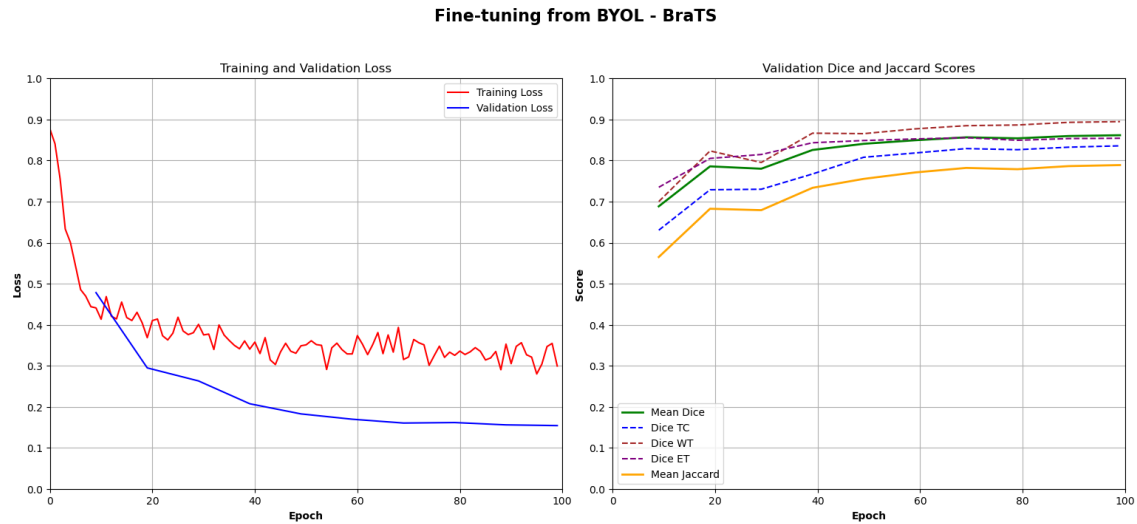


FIGURE 23 Fine-tuning of the BYOL pre-trained model for full BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using the full training data. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset.

For BraTS fine-tuning from BYOL pre-training, the training and validation losses have a steeper decrease at the initial epochs, to then stabilize at around 0.3 for the training loss and 0.15 for the validation loss. The Dice scores and Jaccard index increase slightly across the epochs, reaching values of around 0.85-0.9 for the Dice scores and 0.8 for the Jaccard index.

Fine-tuning from BYOL 10% data - BraTS

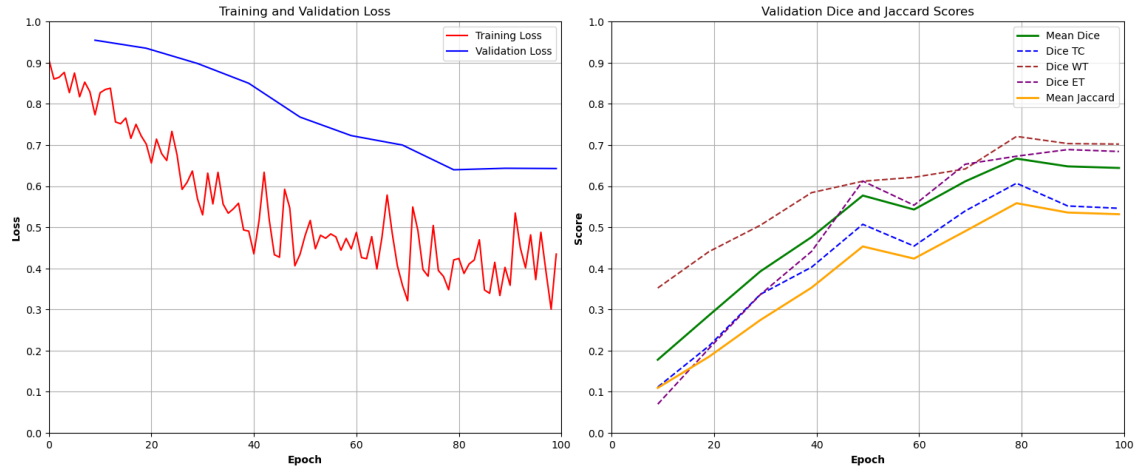


FIGURE 24 Fine-tuning of the BYOL pre-trained model for reduced BraTS data. The graph on the left shows the training and validation losses for the BraTS dataset using 10% of the training split. The graph on the right shows the Dice scores for each segmentation class, the average Dice score and the average Jaccard index across the 10 validation checkpoints for the BraTS dataset.

When reducing the training data for BraTS BYOL pre-trained fine-tuning to 10% of the original amount, the training and validation losses obtained start at an initial value of 0.9 for the training loss and 0.95 for the validation loss, gradually decreasing until reaching a value of 0.4 for the training loss and 0.65 for the validation loss. The Dice scores and Jaccard index start at low values, increasing gradually until epoch 80, where they achieve a relatively stable value of around 0.55-0.7 for the Dice scores and 0.55 for the Jaccard index.

Fine-tuning from BYOL - Selma

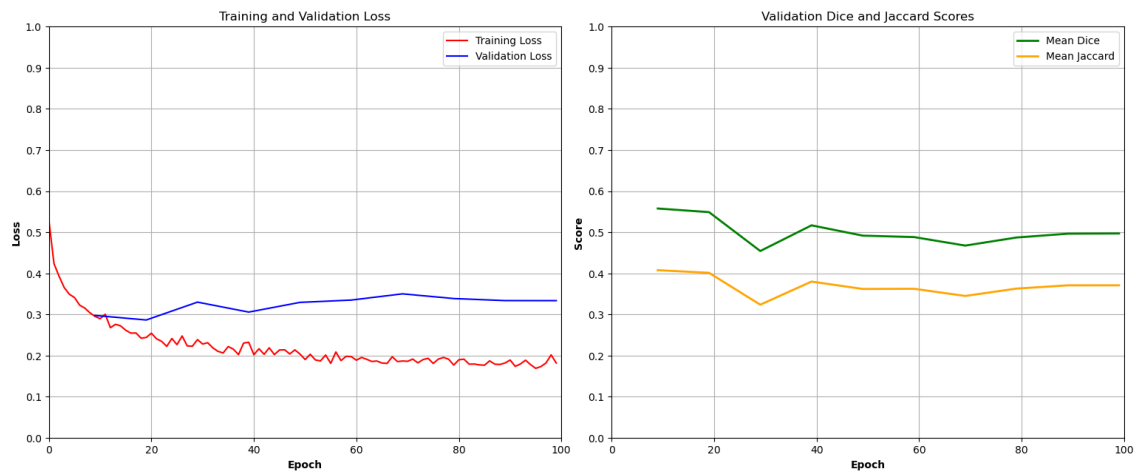


FIGURE 25 Fine-tuning of the BYOL pre-trained model for full Selma data. The graph on the left shows the training and validation losses for the Selma dataset using the full annotated data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset.

For Selma fine-tuning from BYOL pre-training, the training loss also follows a steeper decrease at the initial epochs, to then stabilize at around 0.2 at later epochs. For the

validation loss, no decrease is observed; the loss value is maintained at approximately 0.33 throughout the whole training, with small fluctuations up and down. The Dice score and Jaccard index also have no apparent increase across the validation checks, in fact they decrease slightly from 0.55 and 0.4 respectively to 0.5 and 0.38 values at the end.

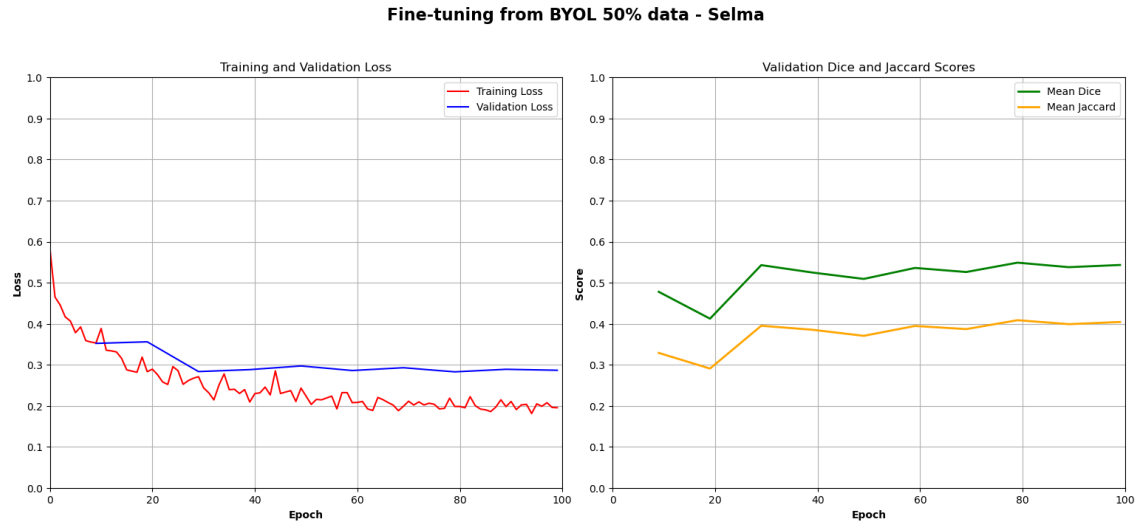


FIGURE 26 Fine-tuning of the BYOL pre-trained model for reduced Selma data. The graph on the left shows the training and validation losses for the Selma dataset using 50% of the training data split. The graph on the right shows the average Dice score and the average Jaccard index across the 10 validation checkpoints for the Selma dataset.

When reducing the training data for Selma BYOL pre-trained fine-tuning to 50% of the original amount, the training and validation losses obtained start at an initial value of 0.57 and 0.35 respectively, with the training loss gradually decreasing until reaching a value of 0.2 and the validation loss having a slight decrease to a value of 0.3. The Dice score and Jaccard index gradually increase across the validation checks, obtaining a relatively stable value of 0.54 and 0.4 respectively after epoch 60.

4.6 Testing results

To test the real-life performance and generalization capabilities of the models, they were tested on their respective test dataloaders using the testing script explained in the previous section. For Selma, the test dataloader contains 7 vessel samples, 2 nuclei samples and 3 cell samples. In the table below, the testing metrics for each model trained for the Selma dataset are shown. These include the average Dice scores obtained for the segmentation of each entity, as well as the mean Dice and test loss for all entities. The values are shown as mean +/- standard deviation, for more detailed performance measure.

Training method	Dice cells	Dice nuclei	Dice vessels	Mean Dice	Test loss
Supervised	0.69 ± 0.02	0.74 ± 0.02	0.39 ± 0.19	0.52 ± 0.21	0.30
Supervised 50%	0.69 ± 0.02	0.72 ± 0.09	0.53 ± 0.24	0.60 ± 0.20	0.26
Fine-tuning MIM	0.70 ± 0.07	0.75 ± 0.06	0.32 ± 0.12	0.49 ± 0.22	0.32
Fine-tuning MIM 50%	0.50 ± 0.1	0.71 ± 0.07	0.55 ± 0.20	0.57 ± 0.17	0.27
Fine-tuning BYOL	0.71 ± 0.02	0.71 ± 0.04	0.49 ± 0.23	0.58 ± 0.21	0.28
Fine-tuning BYOL 50%	0.71 ± 0.01	0.73 ± 0.08	0.44 ± 0.2	0.55 ± 0.21	0.28

TABLE 3 Testing results for the different trained Selma models. The Dice scores for each entity are shown, as well as the average Dice and test loss for all samples. Best performing models in each metric are marked in bold.

For BraTS, the test dataloader contains 38 samples, each with four channels. In the table below, the testing metrics for each model trained for the BraTS dataset are shown. These include the average Dice scores obtained for the segmentation of each class, as well as the mean Dice and test loss for all classes. During training, the Dice scores for each class are aggregated, so the standard deviation for each class is not available.

Training method	Dice TC	Dice WT	Dice ET	Mean Dice	Test loss
Supervised	0.83	0.90	0.85	0.86 ± 0.15	0.16
Supervised 10%	0.66	0.74	0.69	0.69 ± 0.27	0.51
Fine-tuning MIM	0.80	0.88	0.82	0.83 ± 0.18	0.18
Fine-tuning MIM 10%	0.65	0.74	0.69	0.69 ± 0.27	0.49
Fine-tuning BYOL	0.82	0.90	0.83	0.85 ± 0.17	0.17
Fine-tuning BYOL 10%	0.63	0.73	0.69	0.68 ± 0.27	0.54

TABLE 4 Testing results for the different trained BraTS models. The Dice scores for each class are shown, as well as the average Dice and test loss for all samples. Best performing models in each metric are marked in bold.

5 DISCUSSION

Many experiments were conducted to analyze in depth the impact of different SSL strategies in the downstream segmentation task, for both BraTS and Selma. The good performance of the supervised training from scratch in BraTS serves as a sanity check that the architecture is well-made and works properly; the test results shown in Table 4 for the supervised model are consistent with those published in papers [84], albeit slightly lower. However, this can be explained by the difference in computational resources and used training data, as only one fold of the five available was used for supervised training in comparison to studies where all folds are used, and the training only lasted for 100 epochs.

One notable finding is that supervised training in the Selma dataset achieves better performance when only half of the training data is used, instead of the full split. In BraTS, this does not happen, in fact the reduced-data supervised training is considerably worse than the one using the full split, as expected. This trend can also be observed in the fine-tuning experiments, with Selma models using half the available annotated training data outperforming the ones using the full data in the validation metrics and loss, while BraTS models that use the full split significantly outperform the ones that use the reduced split.

In the testing experiments, the division is not as clear-cut, with some Selma models using the full data sometimes performing better than the reduced data models. The fact that this trend only happens in Selma and not in BraTS, and is especially notable during training but not as much during testing, suggests that there are some training samples that disrupt the model’s learning but contain valuable information that can potentially help the model generalize to previously unseen samples. One possible explanation is that the full dataset contains more samples that are significantly harder to segment compared to the more restricted split, causing the model to focus on trying to learn those samples and overfitting them, therefore neglecting to learn useful general representations and underperforming during validation. If the validation set does not contain samples like this, therefore being more similar to the restricted dataset, it makes sense the 50% data model performs better. However, it is possible the testing split contains some of these harder samples, where the model’s adaptation to them is therefore beneficial and therefore sometimes outperforms the restricted dataset model.

In general, it can be observed that the validation metrics for Selma are generally poor no matter the dataset size or training approach used, remaining almost flat during the whole training, while the training loss follows a normal decreasing curve. This suggests that the dataset used for training is too limited, so the model just overfits and does not learn useful features for generalized performance. This is likely exacerbated by the fact that the Selma dataset contains three different entities, where vessels are significantly different in morphology to cells and nuclei, making it even harder for the model to learn generalizable features for all from such a limited annotation pool. Even then, the models trained for SELMA in this thesis, both supervised and pre-trained, obtain Dice scores similar to those of the official Selma3D challenge in cell segmentation, with the best challenge team

obtaining a Dice score of 65.37 ± 13.38 in the final test for c-Fos+ cell segmentation [5]. Compared to the Dice scores obtained for cell segmentation in Table 3, it can be seen that the thesis models are usually on the range of 69-71 score, with much smaller standard deviations. However, it must be noted that although the tested entity is the same, the patches used for testing in the challenge are different to the ones used in this thesis, so a true comparison cannot be made. The performances for the other entities cannot be compared at all, as the challenge paper does not provide test results for the nuclei and vessel segmentation.

Another notable and surprising finding is that neither BYOL nor MIM pre-training seem to have much consistent impact on the downstream task compared to supervised training from scratch. This does not seem to be due to the pre-training being wrongly done, as the pre-training experiments have standard loss curves, with a gradually declining curve; in the case of MIM pre-training it can be observed in the reconstruction visualizations that the model captures well the spatial features and is capable of achieving high-quality reconstructions, with the speckles appearing in the figure being due to the model only learning from masked patches and therefore outputting wrong predictions in the visible patches.

During training, the supervised approach for BraTS with full data slightly outperforms the MIM and BYOL pre-trained models in the end, and in the low-data regime all three models converge to very similar performance. For Selma, the supervised approach with half the data outperforms the pre-trained approaches fine-tuned on half the data, while for the full datasets the BYOL pre-trained model achieves a notably higher mean Dice score than others. It can be observed that at the beginning of the training, the MIM pre-trained model achieves better validation values than the supervised approach for BraTS, and something similar happens with the BYOL pre-trained model and the supervised approach for Selma with all data.

This suggests that some valuable feature learning is preserved that makes a difference at the beginning of training compared to supervised training from scratch. However, the fact that it does not really make a difference as training progresses implies that the features learned by pre-training can be learned just as well from the small, annotated dataset, or that they are not as valuable for segmentation. SSL has been shown to consistently improve downstream performance when done in the right conditions, but most of the times the pre-training dataset is significantly larger than what was used in this thesis, with tens or hundreds of thousand unannotated samples and very long training times. For BYOL specifically, batch sizes of a few thousand are usually used to improve learning, while in this thesis the maximum batch size possible due to computational constraints was 2, with an effective batch size of 16 by using gradient accumulation. The low quantity of pre-training samples for both datasets, and few time and computational resources in comparison to other published studies are the most likely causes as to why the pre-training was insufficient and did not cause an impact in the downstream task.

Even though no significant impact was observed by pre-training on the downstream task, other insights could be extracted from the experiments conducted. In table 3, it can be

seen that the BYOL pre-trained model outperformed the MIM pre-trained model and the supervised model for Selma cell segmentation in both data regimes, and was the second-best model for general segmentation. When using the full training split for Selma, the BYOL pre-trained model also outperforms the MIM pre-trained model in segmentation of all entities except for nuclei segmentation. A curious observation is that for the half data regime, the MIM-pretrained model achieves the highest vessel segmentation score among all but the worst nuclei score, while for the full data split, it achieves the worst score in vessels but the best in nuclei segmentation.

It can also be observed that vessels are the most difficult entity to segment, with their Dice scores being low compared to cells and nuclei and having larger standard deviations than them. The low Dice scores and large standard deviation for vessels suggest that in the test split vessel appearance is diverse, while the low standard deviations for cells and nuclei implies they have more stable shapes. It is reasonable to think that this pattern also appears in the training and validation splits, which may be the reason why models trained in the half data regime perform better, because there are less vessel samples which are too variable and difficult to segment, causing the model to be disrupted and overfit.

In general, the testing results indicate that the BYOL pre-training seems better fitted for the Selma dataset when the full data split is used, outperforming the MIM pre-trained model in general segmentation and dominating cell segmentation. It must be noted that the BYOL pre-training for SELMA ran for around half the time of the MIM pre-training, which could explain why the BYOL approach underperforms in the low data regime. For BraTS, BYOL pre-training also seems to slightly outperform the MIM approach, but in general they all perform similarly.

In conclusion, the experiments highlight the importance of having an appropriate, large-scale pre-training for maximizing the impact of SSL pre-training in the downstream task, and it also shows how pre-training may not be necessary or especially useful when the annotated dataset available already contains enough information and diversity to generalize well to unseen samples. For LSFM in specific, it has exposed the necessity of having a more representative annotated dataset, as the current available data is not enough to properly learn features that can be generalized to other data due to the diversity and sparsity of LSFM volumes. The entity-specific metrics give some insights into which approach may be better fitted for a certain task, and the variability of vessels compared to spot-like entities like cells and nuclei poses the question of if it is worth it to try to train one model to segment all entities, or if it would be more efficient and adequate to train one model specifically for the vessel case due to its unique morphology and variability.

BIBLIOGRAPHY

- [1] V. Rani, M. Kumar, A. Gupta, M. Sachdeva, A. Mittal, and K. Kumar, "Self-supervised learning for medical image analysis: a comprehensive review," *Evol. Syst.*, vol. 15, no. 4, pp. 1607–1633, Aug. 2024, doi: 10.1007/s12530-024-09581-w.
- [2] D. B. Schmolze, C. Standley, K. E. Fogarty, and A. H. Fischer, "Advances in Microscopy Techniques," *Arch. Pathol. Lab. Med.*, vol. 135, no. 2, pp. 255–263, Feb. 2011, doi: 10.5858/135.2.255.
- [3] O. Krupa *et al.*, "NuMorph: Tools for cortical cellular phenotyping in tissue-cleared whole-brain images," *Cell Rep.*, vol. 37, no. 2, p. 109802, Oct. 2021, doi: 10.1016/j.celrep.2021.109802.
- [4] B. H. Menze *et al.*, "The Multimodal Brain Tumor Image Segmentation Benchmark (BRATS)," *IEEE Trans. Med. Imaging*, vol. 34, no. 10, pp. 1993–2024, Oct. 2015, doi: 10.1109/TMI.2014.2377694.
- [5] Y. Chen *et al.*, "SELMA3D challenge: Self-supervised learning for 3D light-sheet microscopy image segmentation," Jan. 12, 2025, *arXiv*: arXiv:2501.03880. doi: 10.48550/arXiv.2501.03880.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," Feb. 06, 2015, *arXiv*: arXiv:1502.01852. doi: 10.48550/arXiv.1502.01852.
- [7] M. Alloghani, D. Al-Jumeily, J. Mustafina, A. Hussain, and A. J. Aljaaf, "A Systematic Review on Supervised and Unsupervised Machine Learning Algorithms for Data Science," in *Supervised and Unsupervised Learning for Data Science*, M. W. Berry, A. Mohamed, and B. W. Yap, Eds., Cham: Springer International Publishing, 2020, pp. 3–21. doi: 10.1007/978-3-030-22475-2_1.
- [8] X. Liu *et al.*, "Self-Supervised Learning: Generative or Contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023, doi: 10.1109/TKDE.2021.3090866.
- [9] S. Naeem, A. Ali, S. Anam, and M. M. Ahmed, "An Unsupervised Machine Learning Algorithms: Comprehensive Review," *Int. J. Comput. Digit. Syst.*, vol. 13, no. 1, pp. 911–921, Apr. 2023, doi: 10.12785/ijcds/130172.
- [10] V. Rani, S. T. Nabi, M. Kumar, A. Mittal, and K. Kumar, "Self-supervised Learning: A Succinct Review," *Arch. Comput. Methods Eng.*, vol. 30, no. 4, pp. 2761–2775, May 2023, doi: 10.1007/s11831-023-09884-2.
- [11] J. Gui *et al.*, "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 9052–9071, Dec. 2024, doi: 10.1109/TPAMI.2024.3415112.
- [12] L. Qu, S. Liu, X. Liu, M. Wang, and Z. Song, "Towards label-efficient automatic diagnosis and analysis: a comprehensive survey of advanced deep learning-based weakly-supervised, semi-supervised and self-supervised techniques in histopathological image analysis," *Phys. Med. Biol.*, vol. 67, no. 20, p. 20TR01, Oct. 2022, doi: 10.1088/1361-6560/ac910a.
- [13] J. E. van Engelen and H. H. Hoos, "A survey on semi-supervised learning," *Mach. Learn.*, vol. 109, no. 2, pp. 373–440, Feb. 2020, doi: 10.1007/s10994-019-05855-6.

- [14] Z.-H. Zhou, “A brief introduction to weakly supervised learning,” *Natl. Sci. Rev.*, vol. 5, no. 1, pp. 44–53, Jan. 2018, doi: 10.1093/nsr/nwx106.
- [15] Z. Ren, S. Wang, and Y. Zhang, “Weakly supervised machine learning,” *CAAI Trans. Intell. Technol.*, vol. 8, no. 3, pp. 549–580, 2023, doi: 10.1049/cit2.12216.
- [16] X. Yang, X. He, Y. Liang, Y. Yang, S. Zhang, and P. Xie, “Transfer Learning or Self-supervised Learning? A Tale of Two Pretraining Paradigms,” Jun. 19, 2020, *arXiv*: arXiv:2007.04234. doi: 10.48550/arXiv.2007.04234.
- [17] S. Albelwi, “Survey on Self-Supervised Learning: Auxiliary Pretext Tasks and Contrastive Learning Methods in Imaging,” *Entropy*, vol. 24, no. 4, p. 551, Apr. 2022, doi: 10.3390/e24040551.
- [18] T. Brown *et al.*, “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 1877–1901. Accessed: Feb. 10, 2026. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html
- [19] D. Spathis, I. Perez-Pozuelo, L. Marques-Fernandez, and C. Mascolo, “Breaking away from labels: The promise of self-supervised machine learning in intelligent health,” *Patterns*, vol. 3, no. 2, Feb. 2022, doi: 10.1016/j.patter.2021.100410.
- [20] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum Contrast for Unsupervised Visual Representation Learning,” Mar. 23, 2020, *arXiv*: arXiv:1911.05722. doi: 10.48550/arXiv.1911.05722.
- [21] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” Jul. 01, 2020, *arXiv*: arXiv:2002.05709. doi: 10.48550/arXiv.2002.05709.
- [22] J. Jang, S. Kim, K. Yoo, C. Kong, J. Kim, and N. Kwak, “Self-Distilled Self-Supervised Representation Learning,” Nov. 24, 2022, *arXiv*: arXiv:2111.12958. doi: 10.48550/arXiv.2111.12958.
- [23] J.-B. Grill *et al.*, “Bootstrap your own latent: A new approach to self-supervised Learning,” Sep. 10, 2020, *arXiv*: arXiv:2006.07733. doi: 10.48550/arXiv.2006.07733.
- [24] M. Caron *et al.*, “Emerging Properties in Self-Supervised Vision Transformers,” May 24, 2021, *arXiv*: arXiv:2104.14294. doi: 10.48550/arXiv.2104.14294.
- [25] X. Chen and K. He, “Exploring Simple Siamese Representation Learning,” Nov. 20, 2020, *arXiv*: arXiv:2011.10566. doi: 10.48550/arXiv.2011.10566.
- [26] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked Autoencoders Are Scalable Vision Learners,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2022, pp. 15979–15988. doi: 10.1109/CVPR52688.2022.01553.
- [27] C. Zhang, C. Zhang, J. Song, J. S. K. Yi, K. Zhang, and I. S. Kweon, “A Survey on Masked Autoencoder for Self-supervised Learning in Vision and Beyond,” Jul. 30, 2022, *arXiv*: arXiv:2208.00173. doi: 10.48550/arXiv.2208.00173.
- [28] Z. Xie *et al.*, “SimMIM: A Simple Framework for Masked Image Modeling,” Apr. 17, 2022, *arXiv*: arXiv:2111.09886. doi: 10.48550/arXiv.2111.09886.

- [29] H. Bao, L. Dong, S. Piao, and F. Wei, “BEiT: BERT Pre-Training of Image Transformers,” Sep. 03, 2022, *arXiv*: arXiv:2106.08254. doi: 10.48550/arXiv.2106.08254.
- [30] M. Hartley, G. J. Kleywegt, A. Patwardhan, U. Sarkans, J. R. Swedlow, and A. Brazma, “The BioImage Archive – Building a Home for Life-Sciences Microscopy Data,” *J. Mol. Biol.*, vol. 434, no. 11, p. 167505, Jun. 2022, doi: 10.1016/j.jmb.2022.167505.
- [31] M. D. Wilkinson *et al.*, “The FAIR Guiding Principles for scientific data management and stewardship,” *Sci. Data*, vol. 3, no. 1, p. 160018, Mar. 2016, doi: 10.1038/sdata.2016.18.
- [32] J. Moore *et al.*, “OME-NGFF: scalable format strategies for interoperable bioimaging data,” Apr. 13, 2021, *bioRxiv*. doi: 10.1101/2021.03.31.437929.
- [33] U. Sarkans *et al.*, “REMBI: Recommended Metadata for Biological Images—enabling reuse of microscopy data in biology,” *Nat. Methods*, vol. 18, no. 12, pp. 1418–1422, Dec. 2021, doi: 10.1038/s41592-021-01166-8.
- [34] T. Zulueta-Coarasa *et al.*, “MIFA: Metadata, Incentives, Formats and Accessibility guidelines to improve the reuse of AI datasets for bioimage analysis,” *Nat. Methods*, vol. 22, no. 11, pp. 2245–2252, Nov. 2025, doi: 10.1038/s41592-025-02835-8.
- [35] M. Hammer *et al.*, “Towards community-driven metadata standards for light microscopy: tiered specifications extending the OME model,” May 31, 2021, *bioRxiv*. doi: 10.1101/2021.04.25.441198.
- [36] L. Nogueira *et al.*, “Comparison of macroscopic and microscopic (stereomicroscopy and scanning electron microscopy) features of bone lesions due to hatchet hacking trauma,” *Int. J. Legal Med.*, vol. 131, no. 2, pp. 465–472, Mar. 2017, doi: 10.1007/s00414-016-1522-1.
- [37] H. S. Lahoti and S. D. Jogdand, “Bioimaging: Evolution, Significance, and Deficit,” *Cureus*, vol. 14, no. 9, p. e28923, doi: 10.7759/cureus.28923.
- [38] S. Dayarathna, K. T. Islam, S. Uribe, G. Yang, M. Hayat, and Z. Chen, “Deep learning based synthesis of MRI, CT and PET: Review and analysis,” *Med. Image Anal.*, vol. 92, p. 103046, Feb. 2024, doi: 10.1016/j.media.2023.103046.
- [39] S. P. Power, F. Moloney, M. Twomey, K. James, O. J. O’Connor, and M. M. Maher, “Computed tomography and patient risk: Facts, perceptions and uncertainties,” *World J. Radiol.*, vol. 8, no. 12, pp. 902–915, Dec. 2016, doi: 10.4329/wjrv.v8.i12.902.
- [40] S. B. info@sagebase.org, “Multi-Atlas Labeling Beyond the Cranial Vault - Workshop and Challenge.” Accessed: Feb. 21, 2026. [Online]. Available: <https://www.synapse.org/Synapse:syn3193805>
- [41] S. B. info@sagebase.org, “Brain Tumor Segmentation (BraTS) Challenges.” Accessed: Feb. 21, 2026. [Online]. Available: <https://www.synapse.org/Synapse:syn53708126>
- [42] J. Trotter *et al.*, “Positron Emission Tomography (PET)/Computed Tomography (CT) Imaging in Radiation Therapy Treatment Planning: A Review of PET Imaging Tracers and Methods to Incorporate PET/CT,” *Adv. Radiat. Oncol.*, vol. 8, no. 5, p. 101212, Mar. 2023, doi: 10.1016/j.adro.2023.101212.

- [43] B. Gutiérrez-Medina, “Optical sectioning of unlabeled samples using bright-field microscopy,” *Proc. Natl. Acad. Sci. U. S. A.*, vol. 119, no. 14, p. e2122937119, Apr. 2022, doi: 10.1073/pnas.2122937119.
- [44] D. J. Stephens and V. J. Allan, “Light Microscopy Techniques for Live Cell Imaging,” *Science*, vol. 300, no. 5616, pp. 82–86, Apr. 2003, doi: 10.1126/science.1082160.
- [45] C. J. Peddie *et al.*, “Volume electron microscopy,” *Nat. Rev. Methods Primer*, vol. 2, no. 1, p. 51, Jul. 2022, doi: 10.1038/s43586-022-00131-9.
- [46] R. S. Fischer, Y. Wu, P. Kanchanawong, H. Shroff, and C. M. Waterman, “Microscopy in 3D: a biologist’s toolbox,” *Trends Cell Biol.*, vol. 21, no. 12, pp. 682–691, Dec. 2011, doi: 10.1016/j.tcb.2011.09.008.
- [47] J. M. Girkin and M. T. Carvalho, “The light-sheet microscopy revolution,” *J. Opt.*, vol. 20, no. 5, p. 053002, Apr. 2018, doi: 10.1088/2040-8986/aab58a.
- [48] F. Ghaznavi, A. Evans, A. Madabhushi, and M. Feldman, “Digital Imaging in Pathology: Whole-Slide Imaging and Beyond,” *Annu. Rev. Pathol. Mech. Dis.*, vol. 8, no. 1, pp. 331–359, Jan. 2013, doi: 10.1146/annurev-pathol-011811-120902.
- [49] R. Krishnan, P. Rajpurkar, and E. J. Topol, “Self-supervised learning in medicine and healthcare,” *Nat. Biomed. Eng.*, vol. 6, no. 12, pp. 1346–1352, Dec. 2022, doi: 10.1038/s41551-022-00914-1.
- [50] C. Matsoukas, J. F. Haslum, M. Sorkhei, M. Soderberg, and K. Smith, “What Makes Transfer Learning Work for Medical Images: Feature Reuse & Other Factors,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, New Orleans, LA, USA: IEEE, Jun. 2022, pp. 9215–9224. doi: 10.1109/CVPR52688.2022.00901.
- [51] D. Wolf *et al.*, “Self-supervised pre-training with contrastive and masked autoencoder methods for dealing with small datasets in deep learning for medical imaging,” *Sci. Rep.*, vol. 13, no. 1, p. 20260, Nov. 2023, doi: 10.1038/s41598-023-46433-0.
- [52] V. Bundeale *et al.*, “Evaluating Self-Supervised Learning in Medical Imaging: A Benchmark for Robustness, Generalizability, and Multi-Domain Impact,” Jul. 26, 2025, *arXiv*: arXiv:2412.19124. doi: 10.48550/arXiv.2412.19124.
- [53] C. Zhang, H. Zheng, and Y. Gu, “Dive into the details of self-supervised learning for medical image analysis,” *Med. Image Anal.*, vol. 89, p. 102879, Oct. 2023, doi: 10.1016/j.media.2023.102879.
- [54] Z. Chen *et al.*, “Masked Image Modeling Advances 3D Medical Image Analysis,” Aug. 23, 2022, *arXiv*: arXiv:2204.11716. doi: 10.48550/arXiv.2204.11716.
- [55] A. Taleb *et al.*, “3D Self-Supervised Methods for Medical Imaging,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 18158–18172. Accessed: Feb. 13, 2026. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/d2dc6368837861b42020ee72b0896182-Abstract.html
- [56] J. Cox *et al.*, “BrainSegFounder: Towards 3D foundation models for neuroimage segmentation,” *Med. Image Anal.*, vol. 97, p. 103301, Oct. 2024, doi: 10.1016/j.media.2024.103301.

- [57] Q. Zhou and H. Zou, "A layer-wise fusion network incorporating self-supervised learning for multimodal MR image synthesis," *Front. Genet.*, vol. 13, Aug. 2022, doi: 10.3389/fgene.2022.937042.
- [58] Z. Guo, N. U. Islam, M. B. Gotway, and J. Liang, "Stepwise incremental pretraining for integrating discriminative, restorative, and adversarial learning," *Med. Image Anal.*, vol. 95, p. 103159, Jul. 2024, doi: 10.1016/j.media.2024.103159.
- [59] X.-B. Nguyen, G. S. Lee, S. H. Kim, and H. J. Yang, "Self-Supervised Learning Based on Spatial Awareness for Medical Image Analysis," *IEEE Access*, vol. 8, pp. 162973–162981, 2020, doi: 10.1109/ACCESS.2020.3021469.
- [60] S. Shabani, M. Homayounfar, V. Vardhanabhuti, M.-A. Nikouei Mahani, and M. Koohi-Moghadam, "Self-supervised region-aware segmentation of COVID-19 CT images using 3D GAN and contrastive learning," *Comput. Biol. Med.*, vol. 149, p. 106033, Oct. 2022, doi: 10.1016/j.compbimed.2022.106033.
- [61] A. Kalapos and B. Gyires-Tóth, "Self-supervised Pretraining for 2D Medical Image Segmentation," in *Computer Vision – ECCV 2022 Workshops*, L. Karlinsky, T. Michaeli, and K. Nishino, Eds., Cham: Springer Nature Switzerland, 2023, pp. 472–484. doi: 10.1007/978-3-031-25082-8_31.
- [62] S. Azizi *et al.*, "Big Self-Supervised Models Advance Medical Image Classification," Apr. 01, 2021, *arXiv*: arXiv:2101.05224. doi: 10.48550/arXiv.2101.05224.
- [63] Y. Zhang, J. Kohne, E. Wittrup, and K. Najarian, "Three-Stage Framework for Accurate Pediatric Chest X-ray Diagnosis Using Self-Supervision and Transfer Learning on Small Datasets," *Diagnostics*, vol. 14, no. 15, p. 1634, Jan. 2024, doi: 10.3390/diagnostics14151634.
- [64] M. Shakouri, F. Iranmanesh, and M. Eftekhari, "DINO-CXR: A self supervised method based on vision transformer for chest X-ray classification," Aug. 01, 2023, *arXiv*: arXiv:2308.00475. doi: 10.48550/arXiv.2308.00475.
- [65] M. Gazda, J. Plavka, J. Gazda, and P. Drotár, "Self-Supervised Deep Convolutional Neural Network for Chest X-Ray Classification," *IEEE Access*, vol. 9, pp. 151972–151982, 2021, doi: 10.1109/ACCESS.2021.3125324.
- [66] O. Ciga, T. Xu, and A. L. Martel, "Self supervised contrastive learning for digital histopathology," *Mach. Learn. Appl.*, vol. 7, p. 100198, Mar. 2022, doi: 10.1016/j.mlwa.2021.100198.
- [67] H. Spitzer, K. Kiwitz, K. Amunts, S. Harmeling, and T. Dickscheid, "Improving Cytoarchitectonic Segmentation of Human Brain Areas with Self-supervised Siamese Networks," in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018*, A. F. Frangi, J. A. Schnabel, C. Davatzikos, C. Alberola-López, and G. Fichtinger, Eds., Cham: Springer International Publishing, 2018, pp. 663–671. doi: 10.1007/978-3-030-00931-1_76.
- [68] V. Despotovic *et al.*, "Glioma subtype classification from histopathological images using in-domain and out-of-domain transfer learning: An experimental study," *Heliyon*, vol. 10, no. 5, Mar. 2024, doi: 10.1016/j.heliyon.2024.e27515.
- [69] D. Anand, P. Annangi, and P. Sudhakar, "Benchmarking Self-Supervised Representation Learning from a million Cardiac Ultrasound images," in *2022 44th*

Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC), Jul. 2022, pp. 529–532. doi: 10.1109/EMBC48229.2022.9871511.

- [70] H. Liu, J. Liu, S. Hou, T. Tao, and J. Han, “Perception consistency ultrasound image super-resolution via self-supervised CycleGAN,” *Neural Comput. Appl.*, vol. 35, no. 17, pp. 12331–12341, Jun. 2023, doi: 10.1007/s00521-020-05687-9.
- [71] H. A. Almubarak, “Self-supervised learning for data augmentation in histopathology image segmentation,” *Sci. Rep.*, vol. 15, no. 1, p. 39596, Nov. 2025, doi: 10.1038/s41598-025-23280-9.
- [72] C. Achard *et al.*, “CellSeg3D: self-supervised 3D cell segmentation for light-sheet microscopy,” *eLife*, vol. 13, Dec. 2024, doi: 10.7554/eLife.99848.2.
- [73] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Dec. 10, 2015, *arXiv*: arXiv:1512.03385. doi: 10.48550/arXiv.1512.03385.
- [74] A. Dosovitskiy *et al.*, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” Jun. 03, 2021, *arXiv*: arXiv:2010.11929. doi: 10.48550/arXiv.2010.11929.
- [75] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” May 18, 2015, *arXiv*: arXiv:1505.04597. doi: 10.48550/arXiv.1505.04597.
- [76] A. Vaswani *et al.*, “Attention is All you Need,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2017. Accessed: Feb. 28, 2026. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html
- [77] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy, “Do Vision Transformers See Like Convolutional Neural Networks?,” Mar. 03, 2022, *arXiv*: arXiv:2108.08810. doi: 10.48550/arXiv.2108.08810.
- [78] F. Hörst *et al.*, “CellViT: Vision Transformers for precise cell segmentation and classification,” *Med. Image Anal.*, vol. 94, p. 103143, May 2024, doi: 10.1016/j.media.2024.103143.
- [79] Z. Liu *et al.*, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” Aug. 17, 2021, *arXiv*: arXiv:2103.14030. doi: 10.48550/arXiv.2103.14030.
- [80] H. Cao *et al.*, “Swin-Unet: Unet-like Pure Transformer for Medical Image Segmentation,” May 12, 2021, *arXiv*: arXiv:2105.05537. doi: 10.48550/arXiv.2105.05537.
- [81] Y. Xie, J. Zhang, C. Shen, and Y. Xia, “CoTr: Efficiently Bridging CNN and Transformer for 3D Medical Image Segmentation,” Mar. 04, 2021, *arXiv*: arXiv:2103.03024. doi: 10.48550/arXiv.2103.03024.
- [82] J. Chen *et al.*, “TransUNet: Transformers Make Strong Encoders for Medical Image Segmentation,” Feb. 08, 2021, *arXiv*: arXiv:2102.04306. doi: 10.48550/arXiv.2102.04306.

- [83] A. Hatamizadeh *et al.*, “UNETR: Transformers for 3D Medical Image Segmentation,” Oct. 09, 2021, *arXiv*: arXiv:2103.10504. doi: 10.48550/arXiv.2103.10504.
- [84] A. Hatamizadeh, V. Nath, Y. Tang, D. Yang, H. Roth, and D. Xu, “Swin UNETR: Swin Transformers for Semantic Segmentation of Brain Tumors in MRI Images,” Jan. 04, 2022, *arXiv*: arXiv:2201.01266. doi: 10.48550/arXiv.2201.01266.
- [85] T. Wald *et al.*, “Revisiting MAE pre-training for 3D medical image segmentation,” Apr. 04, 2025, *arXiv*: arXiv:2410.23132. doi: 10.48550/arXiv.2410.23132.
- [86] Z. Xu *et al.*, “Swin MAE: Masked autoencoders for small datasets,” *Comput. Biol. Med.*, vol. 161, p. 107037, Jul. 2023, doi: 10.1016/j.compbiomed.2023.107037.
- [87] Y. Tang *et al.*, “Self-Supervised Pre-Training of Swin Transformers for 3D Medical Image Analysis,” Mar. 28, 2022, *arXiv*: arXiv:2111.14791. doi: 10.48550/arXiv.2111.14791.
- [88] J. Nilsson and T. Akenine-Möller, “Understanding SSIM,” Jun. 29, 2020, *arXiv*: arXiv:2006.13846. doi: 10.48550/arXiv.2006.13846.
- [89] V. N. Tran *et al.*, “Multi-view and multi-augmentation for self-supervised visual representation learning,” *Appl. Intell.*, vol. 54, no. 1, pp. 629–656, Jan. 2024, doi: 10.1007/s10489-023-05163-6.

APPENDIX A: SOFTWARE AND HARDWARE SPECIFICATIONS

Hardware

The training and testing runs were performed on a cluster managed by SLURM.

- GPU: one NVIDIA A30
- RAM: 64 GB
- CPUS: 8 cores per task

The visualizations were run on my personal laptop.

- Acer Predator PH315-53
- NVIDIA GeForce RTX 3060 Laptop
- VRAM: 6 GB
- CPU: Intel Core i7-10750H
- RAM: 32 GB

Software

- Python 3.10
- PyTorch 2.6.0+cu126
- PyTorch Lightning 2.5.2
- MONAI 1.5.0
- Numpy, SciPy, pandas, scikit-learn, matplotlib
- Mixed precision training (16-bit) via PyTorch Lightning

Code

The code created for this thesis can be accessed at this public GitHub repository:
https://github.com/carla-lm/master_thesis

APPENDIX B: USAGE OF AI DECLARATION

During the writing of the thesis document no AI was used for translation or text generation. The only AI aid used during writing was Word's incorporated spelling and grammar corrector, whose suggestions were taken into account but not always accepted. LLMs were used as support during code development to assist in detecting bugs and in menial coding tasks. The used IDE, PyCharm, has a built-in AI assistant which integrates multiple LLMs (GPT-4o, Gemini, Anthropic Claude). This assistant was used to autocorrect format issues or autocomplete basic coding functions using tab autocomplete; its contributions were small and were always assessed before being accepted.

The LLM models used for more extensive coding support were ChatGPT version GPT-5.2 and Claude version Opus 4.6, whose bigger code contributions are specified in the table below. These models were used to accelerate the process of building basic class structures that can be found in Python and PyTorch documentation, manage folder paths or to aid in writing repetitive plotting code that can be found in any coding aid webpage. However, they were not used to generate and guide fundamental coding decisions. All ideas about what algorithms to use, how the training pipeline should be, experiment design, conclusions, how the visualizations are structured etc were all the author's own work. All assistance in code offered by AI models is reviewed, formatted and cleaned by the author before being included in the script. The comments included in the code are written by the author for clarity, they are not AI generated.

Code generated with AI assistance	Prompt
The code for managing the file paths during data loading was created with the assistance of GPT-5.2.	"My data files for <dataset> are organized in folders in the following manner: ... Can you help me write the recursive code to explore these folders and save these paths for data loading?"
The code for plotting the metrics from the saved csv file during training was created with the assistance of Claude Opus 4.6.	"The metrics for my training runs are saved in the metrics.csv file inside <folder>. Its contents are saved in the following manner:... Can you help me write the code to plot the metrics so they look like <description of how the author wanted to plot the metrics>"
The code for creating and defining the checkpoint paths was created with the assistance of GPT-5.2.	"I want to save my training checkpoints in the following way:...Can you help me write the code to automatically name and create the checkpoint folders in the way I described?"

The hand-signed AI usage declaration is attached below as required.